IT PROJECT REPORT

ON

STUDENT INDUSTRIAL WORK EXPERIENCE SCHEME (SIWES)

DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED LECTURE NOTES AND TEXTBOOK RECOMMENDATION WEB APPLICATION USING NEXT.JS

UNDERTAKEN AT

ACADEVA LIMITED

Akwa Ibom State, Nigeria

BY

OKON UKPEME EZEKIEL

22/SC/CO/1235

SUBMITTED TO

DEPARTMENT OF COMPUTER SCIENCE,
FACULTY OF COMPUTING,
UNIVERSITY OF UYO, UYO.

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE BACHELOR OF SCIENCE (B.Sc.) DEGREE IN COMPUTER SCIENCE

OCTOBER, 2026

TITLE PAGE

DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED LECTURE NOTES AND TEXTBOOK RECOMMENDATION WEB APPLICATION USING NEXT.JS

BY

OKON UKPEME EZEKIEL

22/SC/CO/1235

A REPORT ON THE STUDENT INDUSTRIAL WORK EXPERIENCE SCHEME (SIWES) UNDERTAKEN AT ACADEVA LIMITED, AKWA IBOM STATE, FROM MAY 2026 TO OCTOBER 2026

SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE, FACULTY OF COMPUTING, UNIVERSITY OF UYO, UYO

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE BACHELOR OF SCIENCE (B.Sc.) DEGREE IN COMPUTER SCIENCE

OCTOBER, 2026

CERTIFICATION

This is to certify that this SIWES IT project report, titled "Design and Implementation of an AI-Assisted Lecture Notes and Textbook Recommendation Web Application Using Next.js," was written by **OKON UKPEME EZEKIEL** with matriculation number **22/SC/CO/1235** of the Department of Computer Science, Faculty of Computing, University of Uyo, and is a true record of the industrial training undertaken at Acadeva Limited from May 2026 to October 2026.

It has been read and approved as meeting the requirements of the Students Industrial Work Experience Scheme for the award of the Bachelor of Science (B.Sc.) degree in Computer Science.

Industry Supervisor John Henry, Acadeva Limited	**Signature / Date**
SIWES Coordinator Department of Computer Science	**Signature / Date**
Head of Department Department of Computer Science	**Signature / Date**

DEDICATION

This report is dedicated to God Almighty, the source of wisdom and strength, and to my family, whose support and encouragement carried me through my studies and this industrial training.

ACKNOWLEDGEMENT

My sincere gratitude goes first to God Almighty for life, good health and the grace to complete this industrial training successfully.

I thank the management and staff of Acadeva Limited for accepting me for the Students Industrial Work Experience Scheme and for giving me the opportunity to work on a real product used by Nigerian students. I am especially grateful to my industry supervisor, Mr. John Henry, for his patient guidance, his timely feedback on my code and his encouragement throughout the Web Development track.

I also appreciate the Head of Department, the SIWES coordinator and all the lecturers of the Department of Computer Science, University of Uyo, whose teaching laid the foundation on which this training was built.

Finally, I thank my parents, siblings and friends for their prayers, financial support and understanding, and my fellow interns for the ideas and discussions that made the placement a rewarding experience.

ABSTRACT

This report documents the six-month Students Industrial Work Experience Scheme (SIWES) undertaken at Acadeva Limited, a Nigerian education technology company that organizes study materials from Nigerian universities by school, department and course. The placement was spent on the company's Web Development track, where practical skills were acquired in React, TypeScript, Next.js, Tailwind CSS, browser media APIs, integration of artificial intelligence (AI) services, consumption of third-party REST APIs, and version control with Git and GitHub.

These skills were applied in an IT project titled "Design and Implementation of an AI-Assisted Lecture Notes and Textbook Recommendation Web Application Using Next.js." The application, named Lecture Follow-Up, allows a student to record a lecture in the browser or upload an audio file. The audio is transcribed by a speech-to-text model (Whisper large-v3), the transcript is rewritten by a large language model (GPT-4o mini) into structured Markdown study notes together with a set of topic search terms, and the search terms are used to retrieve and rank related textbooks from the Acadeva library. Progress is streamed to the browser stage by stage as newline-delimited JSON, so the student sees each step as it happens.

The system was developed incrementally, verified with static analysis, a production build, direct API tests and browser-driven tests on desktop and mobile screen sizes, and evaluated for correctness, security, usability, robustness and maintainability. The report concludes with the lessons learned, the limitations of the current version and recommendations for future work.

Keywords: SIWES, Acadeva, Next.js, React, speech-to-text, large language models, study notes, textbook recommendation.

TABLE OF CONTENTS

Title Page ..... **i**
Certification ..... **ii**
Dedication ..... **iii**
Acknowledgement ..... **iv**
Abstract ..... **v**
Table of Contents ..... **vi**
List of Figures ..... **viii**
List of Tables ..... **ix**
List of Abbreviations ..... **x**
CHAPTER ONE: INTRODUCTION TO SIWES PLACEMENT **1**
1.0 Introduction 1
1.1 Background of the SIWES Programme 1
1.2 Objectives of SIWES 1
1.3 Bodies Involved in the Management of SIWES 2
1.4 About Acadeva Limited 2
1.5 History, Vision, Mission and Core Values 3
1.6 Organizational Structure 4
1.7 Duration and Scope of the Internship 5
1.8 Conclusion 6
CHAPTER TWO: PRACTICAL SKILLS ACQUIRED DURING SIWES **7**
2.0 Introduction 7
2.1 Setting Up a Professional Development Environment 7
2.2 Front-End Development with React and TypeScript 7
2.3 Full Stack Development with Next.js 9
2.4 Styling with Tailwind CSS and shadcn/ui 10
2.5 Working with Audio in the Browser 11
2.6 Streaming Responses over HTTP 13
2.7 Integrating AI Models and Prompt Engineering 14
2.8 Consuming the Acadeva Library API 15
2.9 Version Control with Git and GitHub 16
2.10 Testing, Debugging and Code Quality 16
2.11 Professional and Soft Skills 17
2.12 Challenges Encountered and Solutions 17
2.13 Conclusion 18
CHAPTER THREE: IT PROJECT **19**
3.0 Introduction 19

3.1 Project Overview....19
3.2 Background of the Study....19
3.3 Problem Statement....20
3.4 Aim and Objectives....20
3.5 Significance of the Study....21
3.6 Scope of the Study....21
3.7 Requirements Gathering and Analysis....21
3.8 Methodology....23
3.9 System Design....25
3.10 System Implementation....30
3.11 Testing....38
3.12 Evaluation and Discussion of Results....39
3.13 Recommendations and Future Enhancements....40
3.14 Limitations of the Study....40
3.15 Conclusion....40
REFERENCES....42
APPENDICES....44
Appendix A: Installing and Running the Project....44
Appendix B: Sample Recordings....44
Appendix C: Selected Source Code....45
Appendix D: Additional Screenshots....47

LIST OF FIGURES

Figure 1.1: Simplified organizational structure of Acadeva Limited.....5
Figure 2.1: The Lecture Follow-Up home page, built from reusable React components.....9
Figure 2.2: The Upload tab in dark mode, styled with Tailwind CSS and shadcn/ui.....11
Figure 2.3: Recording in progress, with live waveform, timer, microphone selector, pause and stop.....12
Figure 2.4: A finished recording ready for review and processing.....13
Figure 3.1: Use case diagram of Lecture Follow-Up.....23
Figure 3.2: The iterative cycle applied to each increment.....24
Figure 3.3: Architecture of Lecture Follow-Up.....26
Figure 3.4: Sequence diagram for generating lecture notes.....26
Figure 3.5: State diagram of the processing workflow.....27
Figure 3.6: Flowchart of the book recommendation algorithm.....27
Figure 3.7: The Upload tab with the drag-and-drop area and accepted formats.....31
Figure 3.8: Review card showing an uploaded recording (2 minutes 18 seconds) as a playback waveform.....32
Figure 3.9: Progress tracker while the lecture is being transcribed.....34
Figure 3.10: Progress tracker after transcription and note writing, while related books are being found.....34
Figure 3.11: Completed stages and the rendered study notes (the notes shown are sample content supplied to the interface for illustration).....35
Figure 3.12: Related reading from the Acadeva library, returned by a live search for "software engineering".....36
Figure 3.13: Error panel shown when the transcription service rejects the request.....37
Figure 3.14: Lecture Follow-Up on a phone: home page (left), uploaded recording (centre) and related books (right).....37
Figure D.1: Full results page on desktop: review card, completed stages, notes and related books 47

LIST OF TABLES

Table 1.1: Bodies involved in SIWES and their roles....2
Table 1.2: Main products and services of Acadeva....3
Table 1.3: Core values of Acadeva and what they mean in practice....4
Table 1.4: Summary of training activities by month....5
Table 2.1: Commit history of the Lecture Follow-Up repository....16
Table 2.2: Challenges encountered during the training and how they were resolved....17
Table 3.1: Functional requirements....22
Table 3.2: Non-functional requirements....22
Table 3.3: Use case description for "Generate lecture notes"....23
Table 3.4: Development increments....24
Table 3.5: Technology used....24
Table 3.6: Specification of the POST /api/lecture endpoint....28
Table 3.7: Stage events streamed from POST /api/lecture....28
Table 3.8: Fields of the Book type....28
Table 3.9: Rules in the system instruction for note generation....29
Table 3.10: Main source files and their responsibilities....29
Table 3.11: Environment variables....38
Table 3.12: Test cases and results....38
Table B.1: Sample recordings in the testfiles folder....44

LIST OF ABBREVIATIONS

Table 0.1: Abbreviations used in this report

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CAC	Corporate Affairs Commission
CDN	Content Delivery Network
CSS	Cascading Style Sheets
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ITF	Industrial Training Fund
JSON	JavaScript Object Notation
LLM	Large Language Model
MIME	Multipurpose Internet Mail Extensions
NDJSON	Newline-Delimited JSON
NFR	Non-Functional Requirement
NUC	National Universities Commission
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SIWES	Students Industrial Work Experience Scheme
STT	Speech-to-Text
UI	User Interface
URL	Uniform Resource Locator

CHAPTER ONE

INTRODUCTION TO SIWES PLACEMENT

1.0 Introduction

This chapter introduces the Students Industrial Work Experience Scheme (SIWES) and the organization in which the author's industrial training took place, Acadeva Limited. It explains why the scheme was established, what it aims to achieve and the bodies responsible for it. It then describes the company: its products, history, vision, mission, core values and structure. Finally, it states the duration and scope of the placement and summarizes the activities carried out month by month. Together, these provide the context for the skills discussed in Chapter Two and the IT project presented in Chapter Three.

1.1 Background of the SIWES Programme

The Students Industrial Work Experience Scheme is a skills training programme that forms part of the approved minimum academic standards for many degree and diploma programmes in Nigerian tertiary institutions. It was introduced by the Industrial Training Fund (ITF) in 1973 in response to concerns that graduates of Nigerian institutions lacked the practical skills required by employers. The scheme places students in industries and organizations related to their fields of study for a period of supervised work, so that they can apply classroom knowledge to real problems.

For students of Computer Science, SIWES is the first prolonged contact with the way software is built professionally. It exposes the student to modern tools and frameworks, to collaborative workflows such as code review and version control, to the expectations of real users, and to the discipline of delivering working software within a deadline. The experience helps the student make better choices in the final year of study and prepares them for employment or entrepreneurship after graduation.

1.2 Objectives of SIWES

The main objectives of the scheme, as set out by the Industrial Training Fund, are to:

- provide an avenue for students in tertiary institutions to acquire industrial skills and experience in their course of study;
- prepare students for the work situation they are likely to meet after graduation;
- expose students to work methods and techniques in handling equipment and machinery that may not be available in their institutions;
- make the transition from school to the world of work easier, and enhance students' contacts for later job placement;
- provide students with an opportunity to apply their theoretical knowledge in real work situations, thereby bridging the gap between theory and practice; and
- enlist and strengthen employers' involvement in the entire educational process of preparing graduates for employment in industry.

1.3 Bodies Involved in the Management of SIWES

Several bodies share responsibility for the success of the scheme. Their roles are summarized in Table 1.1.

Table 1.1: Bodies involved in SIWES and their roles

Body	Role
Federal Government	Provides funding for the scheme through the Industrial Training Fund and makes participation by relevant organizations mandatory.
Industrial Training Fund (ITF)	Coordinates the scheme nationally, issues guidelines, approves placements, supervises students and pays allowances.
National Universities Commission (NUC)	Oversees the scheme in universities, approves participating programmes and monitors compliance.
Institutions (University of Uyo)	Establish SIWES units, prepare students, assign academic supervisors, assess logbooks and reports, and grade the training.
Employers (Acadeva Limited)	Accept students, assign industry supervisors, provide meaningful work, and assess the student's performance.
Students	Undertake the training diligently, keep the logbook up to date, obey the rules of the organization and submit a report.

1.4 About Acadeva Limited

Acadeva Limited is an education technology company in Nigeria. Its platform gathers lecture notes, course guides, past questions and other study PDFs from Nigerian universities and arranges them by institution, department and course code, so that a student can find the material for a particular course in a few taps. Materials can be downloaded for offline reading, which matters for students with limited or expensive

data, and shared with classmates. Students can also keep a shared academic timetable with reminders, and use study tools such as quizzes and summaries that are built on top of the library.

The Acadeva apps run on Android, iOS, iPadOS, Windows, macOS and Linux. The company is registered with the Corporate Affairs Commission (CAC) and is based in Akwa Ibom State, Nigeria. Its library is also exposed through a public search interface (api.acadeva.xyz) which other student tools can query, and a content delivery network (cdn.acadeva.xyz) from which books are downloaded. Both services were used directly in the project described in this report.

1.4.1 Products and Services

Table 1.2: Main products and services of Acadeva

Product / service	Description
Study library	Course materials, past questions and PDFs organized by school, department and course code, with ratings and download counts.
Offline downloads	Materials can be saved to the device and read without an internet connection.
Sharing	Materials can be shared with classmates and study partners.
Academic schedule	A shared timetable with reminders for classes, tests and deadlines.
Study tools	Quizzes and summaries generated from the library to support revision.
Quiz events	Large online quiz competitions organized with student communities.
Public search API	An HTTP endpoint that returns books matching a search phrase, used by Acadeva's own tools and partner projects.
Subscriptions	Paid plans that unlock additional features.

1.5 History, Vision, Mission and Core Values

1.5.1 Organizational History

Acadeva began as a prototype that took first and second place at a campus technology competition in November 2024. The Android app was released to the public on 20 June 2025, and more than 2,000 students registered within the six months that followed. In July 2025 the company was named First Runner-Up at DevEast Entrepreneurship Week, held at the Federal University of Technology, Owerri (FUTO), and received startup funding as a result.

Partnerships with student bodies and academic communities have driven the company's growth since then. More than 1,000 books have been shared through its library, and it has organized over 20 large quiz events. Apps for iOS, iPadOS, Windows, macOS and Linux followed the Android release, and paid subscription plans were introduced to support the platform's development.

1.5.2 Vision Statement

To make quality academic resources accessible to every student and create a more connected way for students to learn together.

1.5.3 Mission Statement

To make study resources accessible and learning more effective by building collaborative and innovative tools designed around students' real academic needs.

1.5.4 Core Values

Table 1.3: Core values of Acadeva and what they mean in practice

Value	Meaning in practice
Accessibility	Study materials should reach every student, including those with low-end phones and limited data.
Collaboration	Students learn better together, so sharing and group features are central to the product.
Innovation	New technology, such as AI study tools, is adopted when it genuinely helps students learn.
Student-Centricity	Features are designed around real academic needs observed in Nigerian universities.
Integrity	Materials, ratings and results are presented honestly, and users' data is handled responsibly.

1.6 Organizational Structure

Acadeva is a small company with a flat structure. A core team handles engineering, machine learning and design, and a wider group of about nine people contributes to development, design and marketing. Interns are attached to tracks within engineering. The author was placed on the Web Development track under the supervision of John Henry. Figure 1.1 shows a simplified view of the structure.

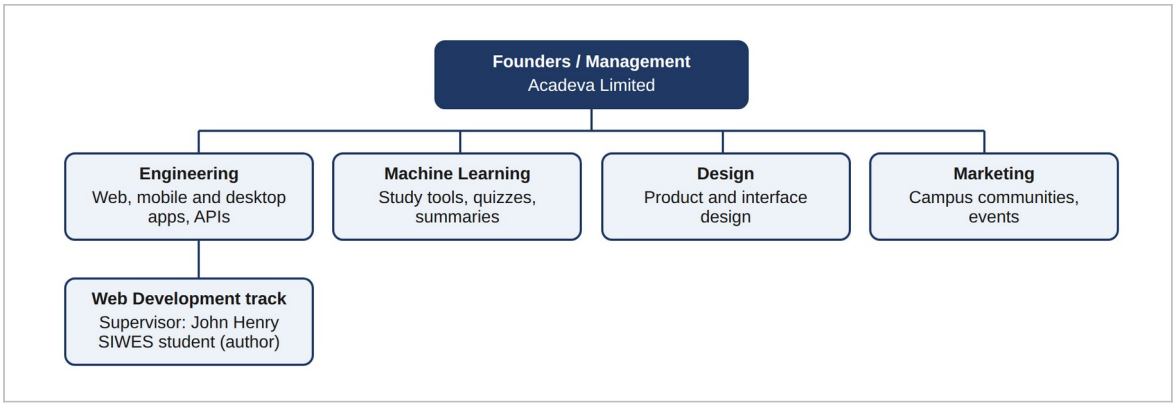


Figure 1.1: Simplified organizational structure of Acadeva Limited

1.7 Duration and Scope of the Internship

The placement ran for six months, from May 2026 to October 2026. It was organized so that the intern could first understand how a product used by thousands of students is designed, built and improved, and then apply that understanding to a project of their own. Work was carried out on the Web Development track, with emphasis on React, TypeScript, Next.js, integration with external web services and AI models, and version control with Git and GitHub.

The activities included guided study of modern front-end and full stack development, practice exercises on components, routing, styling, browser media APIs and HTTP requests, and the design, implementation, testing and documentation of an IT project: Lecture Follow-Up, a web application that turns a recorded or uploaded lecture into structured study notes and recommends related textbooks from the Acadeva library. Table 1.4 summarizes the activities by month.

Table 1.4: Summary of training activities by month

Month	Main activities
May 2026	Orientation and introduction to the Acadeva product and team; setting up the development environment (Node.js, pnpm, VS Code, Git); refresher on modern JavaScript and introduction to TypeScript.
June 2026	React fundamentals: components, props, state and hooks; building small practice interfaces; reading and following the official documentation.
July 2026	Next.js App Router, layouts and route handlers; styling with Tailwind CSS and shadcn/ui; responsive layouts for phones.
August 2026	Consuming REST APIs, including the Acadeva search API; browser audio (MediaRecorder, File API) and waveform display; introduction to AI APIs and structured output.
September 2026	Design and implementation of the Lecture Follow-Up project: recording and upload, transcription, note generation, book recommendation and

Month	Main activities
	streamed progress; source code pushed to GitHub.
October 2026	Testing and evaluation of the project, documentation (README) and preparation of this report.

1.8 Conclusion

This chapter described the SIWES programme, its objectives and the bodies that manage it, introduced Acadeva Limited, its products, history, values and structure, and set out the duration and scope of the placement. The next chapter presents the practical skills acquired during the training.

CHAPTER TWO

PRACTICAL SKILLS ACQUIRED DURING SIWES

2.0 Introduction

This chapter describes the practical skills gained during the placement. For each skill it explains what the technology is and why it matters, then describes how it was used, with the Lecture Follow-Up project as the running example. Short excerpts from the project's source code are included where they make the explanation clearer. The chapter closes with the challenges met during the training and how they were resolved.

2.1 Setting Up a Professional Development Environment

Before any code is written, a developer needs a reliable environment: a runtime to execute JavaScript outside the browser, a package manager to install libraries, an editor with good language support, and version control. Setting this up correctly, and in the same way as the rest of the team, avoids the common problem of code that works on one computer but not another.

- **Practical activity:** Installed Node.js 22 and the pnpm package manager, configured Visual Studio Code with TypeScript, ESLint and Tailwind CSS extensions, configured Git with name and email, and learned the main pnpm commands used in the project (pnpm install, pnpm dev, pnpm build, pnpm start and pnpm lint). Learned to keep secrets such as API keys in a .env file that is excluded from Git.
- **Tools/technologies used:** Node.js 22, pnpm 10, Visual Studio Code, Git, terminal.
- **Skills acquired:** Environment configuration, dependency management, use of scripts, and secure handling of configuration files.

2.2 Front-End Development with React and TypeScript

React is a JavaScript library for building user interfaces from components: small, self-contained pieces of the interface that receive data through props and keep their own state. Hooks such as useState, useEffect, useRef and useCallback let a component store values, react to changes and clean up after itself. TypeScript adds static types to JavaScript, so that mistakes such as passing the wrong kind of data between components are caught before the code runs.

- **Practical activity:** Built the Lecture Follow-Up interface as a set of typed React components: a recorder panel with Record and Upload tabs, a live recording waveform, a playback waveform, a processing-stages tracker, a Markdown notes viewer and a grid of book cards. The page component holds the overall state (idle, transcribing, explaining, finding books, done or error) and passes it down to the components that display it. Shared types such as Book and LectureResult are defined once and used by both the browser and server code.
- **Tools/technologies used:** React 19, TypeScript 5, lucide-react icons, browser developer tools.
- **Skills acquired:** Component design, props and state, hooks, typed interfaces, conditional rendering and handling user events.

Listing 2.1: The processing stages and their labels (components/lecture/processing-stages.tsx)

```
export type ProcessingStage =
  | "idle"
  | "transcribing"
  | "explaining"
  | "finding-books"
  | "done"
  | "error"

interface ProcessingStagesProps {
  stage: ProcessingStage
  errorMessage?: string
}

type StepStatus = "pending" | "active" | "done"

const STEPS: { key: string; label: string }[] = [
  { key: "transcribing", label: "Transcribing audio" },
  { key: "explaining", label: "Writing lecture notes" },
  { key: "finding-books", label: "Finding related books" },
]
```

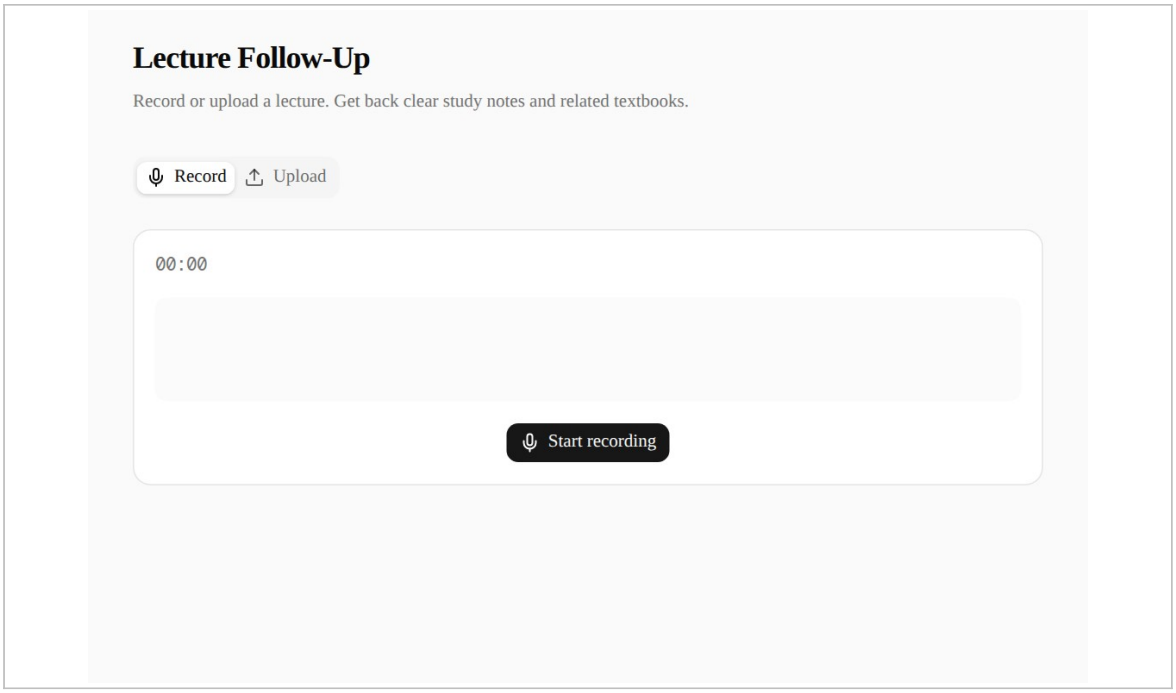


Figure 2.1: The Lecture Follow-Up home page, built from reusable React components

2.3 Full Stack Development with Next.js

Next.js is a framework built on React that adds routing, server rendering and back-end endpoints to the same project. With the App Router, each folder under app/ becomes a route, and a file named route.ts defines a route handler that responds to HTTP requests on the server. Secrets such as API keys stay on the server, while the browser receives only the results. Next.js also produces an optimized production build of the whole application.

- **Practical activity:** Created the main page in app/page.tsx and a route handler at app/api/lecture/route.ts that accepts a POST request containing an audio file. The handler checks the audio format, converts the file to base64, calls the transcription and note-writing services, searches for books, and streams progress back to the browser. The project was built for production with next build and served with next start. Because the installed Next.js version differs from older versions, the framework's own bundled documentation was read before writing code, as the project's AGENTS.md file instructs.
- **Tools/technologies used:** Next.js 16 (App Router), Node.js 22, pnpm, environment variables.
- **Skills acquired:** Routing, client and server code, route handlers, request validation, environment configuration and production builds.

Listing 2.2: Resolving the audio format from the MIME type or file extension (app/api/lecture/route.ts)

```
function resolveFormat(mimeType: string, filename: string): SttFormat | null {
  const normalizedMime = mimeType.split(";")[0].trim().toLowerCase();
  if (MIME_TO_FORMAT[normalizedMime]) {
    return MIME_TO_FORMAT[normalizedMime];
  }

  const ext = filename.split(".").pop()?.toLowerCase();
  if (ext && EXTENSION_TO_FORMAT[ext]) {
    return EXTENSION_TO_FORMAT[ext];
  }

  return null;
}
```

2.4 Styling with Tailwind CSS and shadcn/ui

Tailwind CSS is a utility-first CSS framework: instead of writing separate style sheets, small classes such as flex, gap-4 or text-sm are applied directly to elements. shadcn/ui is a collection of accessible, well-designed components (buttons, tabs, cards, badges, progress bars) whose source code is copied into the project so it can be adapted freely. Together they make it quick to build a clean, consistent and responsive interface.

- **Practical activity:** Used shadcn/ui components for the tabs, buttons, cards, badges, progress bar and toasts, and Tailwind classes for layout and spacing. The Tailwind Typography plugin was used to style the generated Markdown notes, and responsive classes such as grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 make the book grid adapt to the width of the screen. The interface also follows the system's dark mode automatically.
- **Tools/technologies used:** Tailwind CSS 4, shadcn/ui, @tailwindcss/typography, next-themes, Sonner.
- **Skills acquired:** Utility-first styling, component libraries, responsive design and consistent visual design.

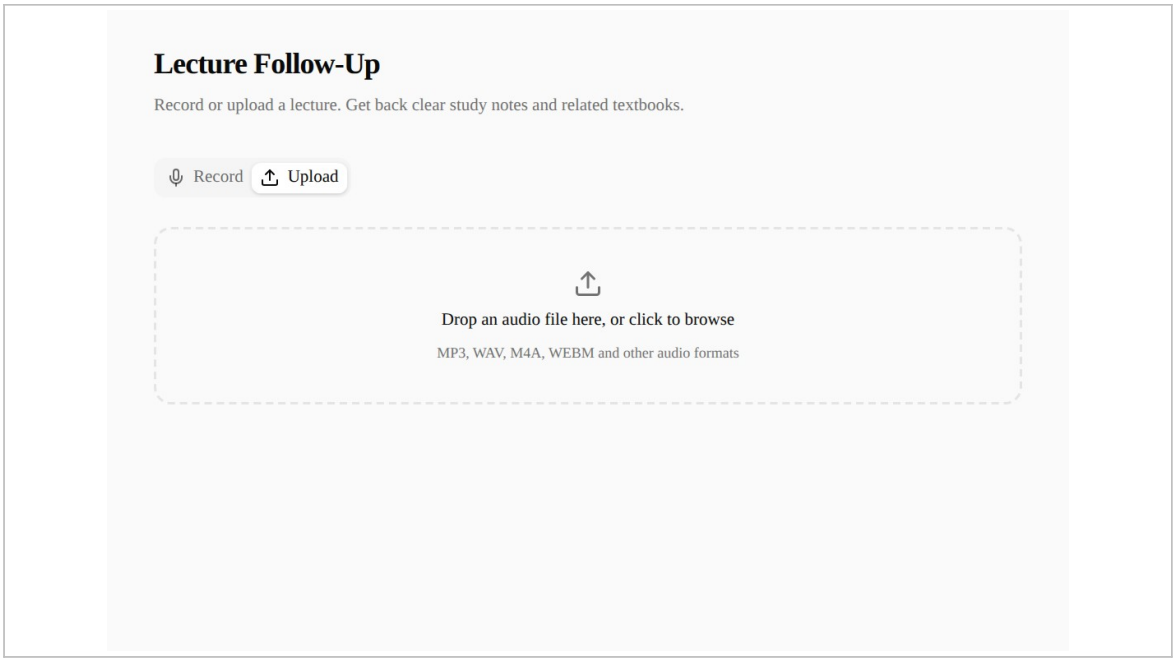


Figure 2.2: The Upload tab in dark mode, styled with Tailwind CSS and shadcn/ui

2.5 Working with Audio in the Browser

Modern browsers can capture sound from a microphone through the MediaDevices and MediaRecorder APIs, and can read files chosen by the user through the File API. Displaying audio as a waveform gives users visual feedback that recording is working and lets them check a file before sending it. Different browsers record in different formats, so an application must detect and handle the format it receives.

- **Practical activity:** Used wavesurfer.js and its Record plugin to show a live, scrolling waveform while recording, with a choice of microphone, a running timer, and pause, resume and stop controls. Built an upload area that accepts files by clicking or by drag-and-drop, rejects files that are not audio, and shows a playback waveform and the duration before processing. Object URLs created for previews are revoked when no longer needed, so the browser does not keep old recordings in memory.
- **Tools/technologies used:** wavesurfer.js 7 and its Record plugin, MediaRecorder API, File API, Blob and object URLs.
- **Skills acquired:** Media capture, file handling, waveform visualization, format detection and cleaning up browser resources.

Listing 2.3: Creating the live recording waveform and listening for recorder events (components/lecture/waveform-recorder.tsx)

```
    const ws = WaveSurfer.create({
      container: containerRef.current,
      waveColor: "rgba(99, 102, 241, 0.5)",
      progressColor: "rgb(99, 102, 241)",
      cursorColor: "transparent",
      height: 80,
      barWidth: 2,
      barGap: 1,
      barRadius: 2,
      normalize: true,
    });

    const record = ws.registerPlugin(
      RecordPlugin.create({
        scrollingWaveform: true,
        renderRecordedAudio: true,
      })
    );

    record.on("record-start", () => {
      setIsRecording(true);
      setIsPaused(false);
      setHasRecording(false);
      setDuration(0);
      onRecordingStateChangeRef.current?.(true);
    });

    record.on("record-progress", (d) => {
```

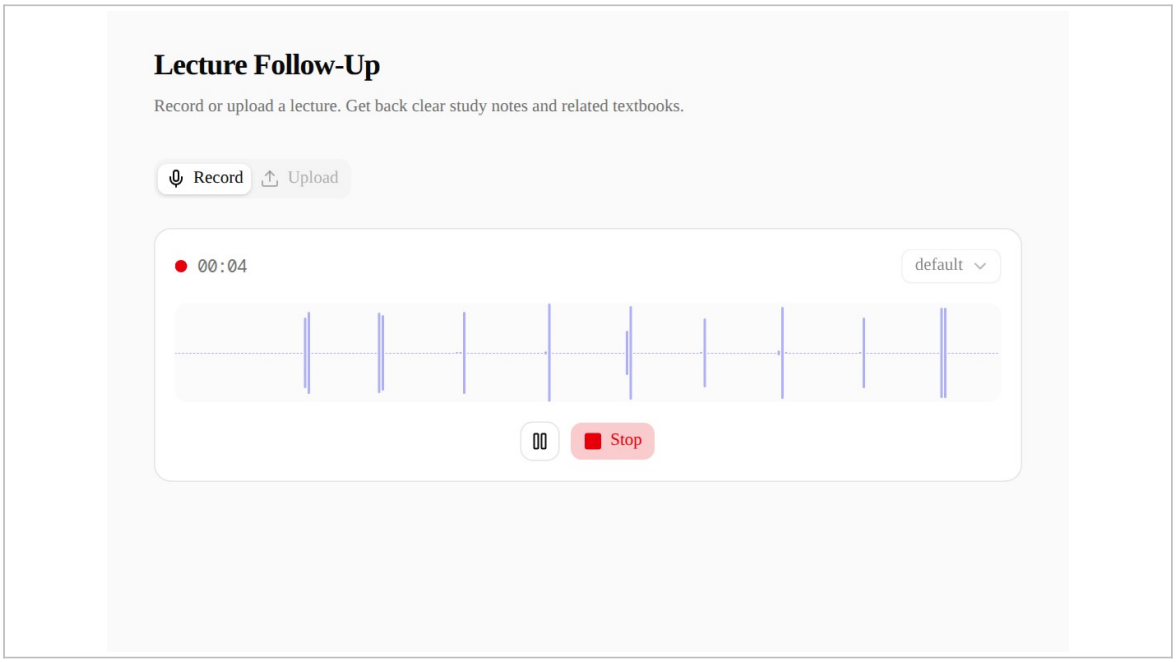


Figure 2.3: Recording in progress, with live waveform, timer, microphone selector, pause and stop

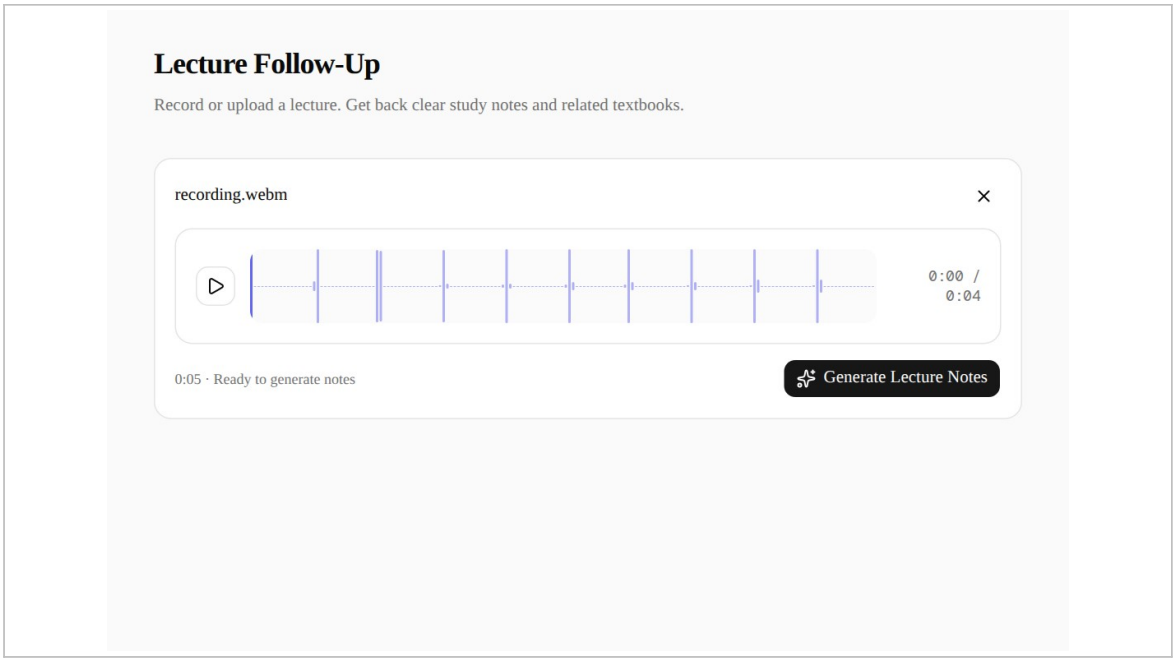


Figure 2.4: A finished recording ready for review and processing

2.6 Streaming Responses over HTTP

A normal HTTP request returns its whole response at once, which is a poor experience when the work takes a minute or more. A streamed response sends data in pieces as it becomes available. Newline-delimited JSON (NDJSON) is a simple streaming format in which each line is a complete JSON object, so the receiver can process each line as soon as it arrives.

- **Practical activity:** Implemented the /api/lecture route handler with a ReadableStream that writes one JSON event per line for each stage of processing. On the browser side, the response body is read with a stream reader and a TextDecoder, the text is split into lines, incomplete lines are kept in a buffer until the rest arrives, and each complete event updates the interface.
- **Tools/technologies used:** ReadableStream, TextEncoder and TextDecoder, Fetch API, NDJSON.
- **Skills acquired:** Streaming, incremental parsing, asynchronous programming and designing a message protocol between client and server.

Listing 2.4: Reading the NDJSON stream in the browser (app/page.tsx)

```
        const reader = res.body.getReader();
        const decoder = new TextDecoder();
        let buffer = "";

        while (true) {
          const { done, value } = await reader.read();
          if (done) break;

          buffer += decoder.decode(value, { stream: true });
          const lines = buffer.split("\n");
          buffer = lines.pop() ?? "";

          for (const line of lines) {
            if (!line.trim()) continue;
            const event = JSON.parse(line) as LectureStageEvent;
```

2.7 Integrating AI Models and Prompt Engineering

Speech-to-text (STT) models convert spoken audio into text, and large language models (LLMs) can read text and produce new, structured text from it. OpenRouter is a service that provides access to many such models through a single HTTP interface. Prompt engineering is the practice of writing clear instructions for a model so that its output is reliable and useful. To make an LLM's reply usable by a program, the request can include a JSON schema that forces the reply into an exact shape.

- **Practical activity:** Wrote lib/openrouter.ts with two functions. transcribeAudio sends the base64 audio to OpenRouter's transcription endpoint (openai/whisper-large-v3 by default). explainLecture sends the transcript to a chat model (openai/gpt-4o-mini by default) with a detailed system instruction and a strict JSON schema, so the reply always contains a Markdown study note and a list of three to six textbook-style search terms. The instruction was refined so that the model explains concepts in its own words instead of returning a cleaned transcript, and phrases search terms the way a university library catalogue would. Both model names can be changed with environment variables.
- **Tools/technologies used:** OpenRouter API, Whisper large-v3, GPT-4o mini, JSON Schema structured output, Fetch API.
- **Skills acquired:** Prompt writing, structured output, calling external REST APIs, error handling and keeping API keys on the server.

Listing 2.5: Requesting structured output with a strict JSON schema (lib/openrouter.ts)

```
      response_format: {
        type: "json_schema",
        json_schema: {
          name: "lecture_explanation",
          strict: true,
          schema: {
            type: "object",
            properties: {
              markdown: { type: "string" },
              searchTerms: { type: "array", items: { type: "string" } },
            },
            required: ["markdown", "searchTerms"],
            additionalProperties: false,
          },
        },
      },
```

2.8 Consuming the Acadeva Library API

Applications often combine data from several services. Calling an external API well means building correct URLs, encoding user input safely, handling failed or slow requests gracefully, and transforming the raw response into the shape the rest of the application needs.

- **Practical activity:** Wrote lib/acadeva.ts, which takes the search terms produced by the language model, removes blanks and duplicates, and queries the Acadeva search endpoint for each term in parallel. Results are merged by book ID, ranked first by how many terms matched and then by rating and downloads, and the top eight are returned. If fewer than five books are found, a combined fallback search is run. Each book is mapped to a cover image URL, a view link and a direct download link. A failed search for one term is logged and skipped rather than stopping the whole request.
- **Tools/technologies used:** Acadeva public books search API, Promise.all, Map and Set, encodeURIComponent, TypeScript interfaces.
- **Skills acquired:** API integration, parallel requests, de-duplication, ranking and defensive error handling.

Listing 2.6: Fetching one search term safely (lib/acadeva.ts)

```
async function fetchTerm(term: string, limit: number): Promise<RawBook[]> {
  try {
    const url = `${ACADEVA_BASE_URL}?q=${encodeURIComponent(term)}&limit=$
{limit}`;
    const res = await fetch(url);
    if (!res.ok) {
      console.error(`Acadeva search failed for term "${term}": ${res.status}`);
      return [];
    }
    const data = (await res.json()) as AcadevaSearchResponse;
    return data.results ?? [];
  } catch (err) {
    console.error(`Acadeva search errored for term "${term}":`, err);
    return [];
  }
}
```

2.9 Version Control with Git and GitHub

Git records the history of a project as a series of commits, so that changes can be traced, compared and undone. GitHub hosts Git repositories online and adds collaboration features such as branches, pull requests and code review, as well as a README file that tells others how to install and run the project.

- **Practical activity:** Created the repository from the Create Next App starter, committed the working application, and added sample lecture recordings for manual testing. The project was pushed to GitHub under the repository lecture-follow-up, and a README was written describing how the application works, how to run it and which environment variables it needs. Work for this report was done on a separate feature branch. Table 2.1 shows the commit history of the application.
- **Tools/technologies used:** Git, GitHub, command line.
- **Skills acquired:** Committing, writing clear commit messages, branching, working with remote repositories and writing project documentation.

Table 2.1: Commit history of the Lecture Follow-Up repository

Commit	Date	Description
91add7a	18 Sep 2026	Initial commit from Create Next App
ff9ddad	18 Sep 2026	First commit: recording, upload, transcription, notes and book search
ca60e02	18 Sep 2026	Add sample audio files (ECEWS report and TechRes admin system) for testing

2.10 Testing, Debugging and Code Quality

Software must be checked before it is trusted. Static analysis tools such as ESLint find likely mistakes without running the code; a production build confirms that the whole

project compiles; and browser automation tools such as Playwright can drive the real interface the way a user would. Debugging is the skill of finding the cause of a problem from its symptoms, using tools such as the browser's developer console and network panel.

- **Practical activity:** Ran ESLint across the project and fixed reported issues; confirmed that next build completes without errors; used curl to send deliberately wrong requests to the API; and used Playwright to open the application in Chromium at desktop size and on an emulated Pixel 7 phone, upload sample recordings, record with a simulated microphone, and capture the screenshots used in this report. The browser's network panel was used to inspect the streamed events during development.
- **Tools/technologies used:** ESLint, Playwright, curl, Chromium developer tools.
- **Skills acquired:** Static analysis, manual and automated testing, debugging and reading error messages carefully.

2.11 Professional and Soft Skills

Alongside the technical work, the placement developed habits that matter in any professional setting:

- **Self-directed learning:** reading official documentation first, especially when a framework version differed from older tutorials.
- **Communication:** reporting progress and blockers clearly to the supervisor and explaining technical decisions in simple terms.
- **Time management:** breaking the project into small increments and finishing each before starting the next.
- **Attention to detail:** checking edge cases such as unsupported files, empty search results and failed network calls.
- **Documentation:** writing a README, commenting non-obvious code and preparing this report.

2.12 Challenges Encountered and Solutions

Table 2.2: Challenges encountered during the training and how they were resolved

Challenge	Solution
Long waits with no feedback while a lecture was processed.	Replaced the single request-and-wait design with a streamed NDJSON response and a stage-by-stage progress tracker.
Language model replies that were not always valid JSON.	Used a strict JSON schema in the request so the reply is always in the expected shape.

Challenge	Solution
Browsers record audio in different formats (webm, ogg, mp4).	Resolved the format on the server from the MIME type with a fallback to the file extension, and rejected unknown formats with a clear message.
Some search terms returned few or no books.	Searched several terms in parallel, merged and ranked the results, and added a combined fallback search when fewer than five books were found.
Framework behaviour differed from older tutorials.	Read the documentation bundled with the installed Next.js version before writing code.
Audio requests rejected by the AI provider when the account balance was too low.	Documented the provider's minimum balance in the README and showed the provider's error message to the user.

2.13 Conclusion

The skills described in this chapter cover the whole path of a modern web feature: capturing input in the browser, processing it on the server with third-party AI services, combining the results with an external library, and presenting them clearly while keeping the user informed. They were applied together in the project presented in the next chapter.

CHAPTER THREE

IT PROJECT: DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED LECTURE NOTES AND TEXTBOOK RECOMMENDATION WEB APPLICATION USING NEXT.JS

3.0 Introduction

This chapter presents the IT project carried out during the placement at Acadeva Limited, titled "Design and Implementation of an AI-Assisted Lecture Notes and Textbook Recommendation Web Application Using Next.js." The application, Lecture Follow-Up, lets a student record a lecture in the browser or upload a recording, and returns organized study notes together with related textbooks from the Acadeva library. The chapter covers the background, problem, objectives, significance, requirements, methodology, design, implementation, testing and evaluation of the system, and ends with recommendations and limitations.

3.1 Project Overview

Lecture Follow-Up is a single-page web application with a small back end. In the browser, the student either records the lecture with a live waveform or uploads an audio file. When the student clicks Generate Lecture Notes, the audio is sent to a Next.js route handler, which passes it through three stages: transcription by a speech-to-text model, note writing by a large language model, and a textbook search on the Acadeva library using topics identified by the model. The server reports each stage to the browser as it starts, so the student sees progress instead of a single spinner. The final result is shown as formatted notes followed by a grid of books that can be viewed or downloaded.

3.2 Background of the Study

Lectures remain the main way university courses are taught in Nigeria, yet students often struggle to take complete notes while also listening and understanding. Large classes, poor acoustics, fast speakers and unfamiliar terms make it easy to miss key points. Many students record lectures on their phones, but recordings are rarely played back because listening again takes as long as the lecture itself, and an hour of audio is hard to search.

Recent advances in automatic speech recognition, such as OpenAI's Whisper model, have made it practical to convert speech to text with good accuracy across accents and recording conditions. Large language models can then rewrite a long, unstructured transcript as clear explanations with headings, definitions and summaries. At the same time, Acadeva has built a large, course-organized library of Nigerian university materials. Connecting these technologies allows a student to go from a lecture they have just attended to clear notes and to the textbooks and course guides that cover the same topics. This fits Acadeva's mission of making study resources accessible and learning more effective, and it exercises the full stack and integration skills learned during the placement.

3.3 Problem Statement

Students often leave lectures with incomplete or disorganized notes and no clear idea of which books cover the material. Raw recordings are long and hard to review, and a plain transcript is unstructured, full of filler words and still not a study note. Finding relevant reading material requires knowing the right search terms, which a student new to a topic may not know. Existing transcription tools stop at the transcript, and general chat assistants do not connect the student to the course materials available in their own university context. There is therefore a need for a simple tool that accepts a lecture recording and returns concise, well-organized notes along with suitable textbooks from a trusted student library.

3.4 Aim and Objectives

Aim: To design, implement and evaluate a web application that converts a recorded or uploaded lecture into structured study notes and recommends related textbooks from the Acadeva library.

Objectives: The specific objectives are:

1. to identify the functional and non-functional requirements of the application;
2. to design the architecture and the flow of data between the browser, the server and the external services;
3. to implement in-browser recording and file upload with waveform feedback using React and wavesurfer.js;
4. to implement transcription and note generation using speech-to-text and large language models through the OpenRouter API;

5. to implement textbook recommendation by querying and ranking results from the Acadeva library API;
6. to give the user continuous feedback by streaming progress from the server; and
7. to manage the source code with Git and GitHub, and to test and evaluate the application.

3.5 Significance of the Study

The project is significant in several ways:

- **For students,** it reduces the effort of turning a lecture into revision material and points them directly to relevant books, which supports better understanding and preparation for examinations.
- **For Acadeva,** it demonstrates a new way of bringing students into the library: from the content of a lecture rather than from a manual search, increasing the value of the materials already collected.
- **For lecturers,** it offers students a way to review difficult lectures without asking for the material to be repeated.
- **For the author,** it provided practical experience of combining a modern web framework with AI services and external APIs in one complete, working product.

3.6 Scope of the Study

The project covers recording and uploading audio, transcription, generation of study notes, textbook recommendation from the Acadeva library and the streamed progress display, on both desktop and mobile browsers. User accounts, saving notes for later, editing notes inside the application, exporting notes to PDF and processing of very long recordings in chunks are outside the scope of this version.

3.7 Requirements Gathering and Analysis

Requirements came from discussions with the supervisor, the author's own experience and that of classmates in taking lecture notes, a review of the Acadeva library and its search interface, and a review of the speech-to-text and language models available through OpenRouter. The requirements are grouped into functional requirements (what the system must do) and non-functional requirements (how well it must do it).

3.7.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	The user can record a lecture in the browser, choose a microphone, pause, resume and stop, with a live waveform and timer.
FR2	The user can upload an audio file (mp3, wav, m4a, ogg, webm, aac or flac) by selecting it or dragging it onto the page.
FR3	The user can play back the captured audio, see its duration, and remove it to start again.
FR4	The system transcribes the audio into text.
FR5	The system produces Markdown study notes with a title, sections, highlighted key terms and a Key Takeaways list.
FR6	The system identifies the lecture's topics and recommends up to eight related books from the Acadeva library, with cover, rating, pages, course code, a view link and a download link.
FR7	The interface shows the progress of each stage (transcribing, writing notes, finding books).
FR8	The interface shows a clear message if any stage fails.
FR9	The interface adapts to phone and desktop screens and to light and dark mode.

3.7.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

ID	Category	Requirement
NFR1	Security	API keys are read only on the server and are never sent to the browser.
NFR2	Validation	Requests without audio or with unsupported formats are rejected before any external call is made.
NFR3	Responsiveness	The user receives feedback within a second of starting and at every stage.
NFR4	Robustness	A failure in the book search does not lose the notes already produced.
NFR5	Configurability	Model names can be changed through environment variables without code changes.
NFR6	Maintainability	External services are isolated in separate modules with typed interfaces.
NFR7	Usability	A lecture can be turned into notes with two actions: capture audio, then Generate.

3.7.3 Use Case Model

Figure 3.1 shows the use case diagram. The student is the only human actor. The three external services act as secondary actors that the system relies on to complete the Generate lecture notes and View / download books use cases.

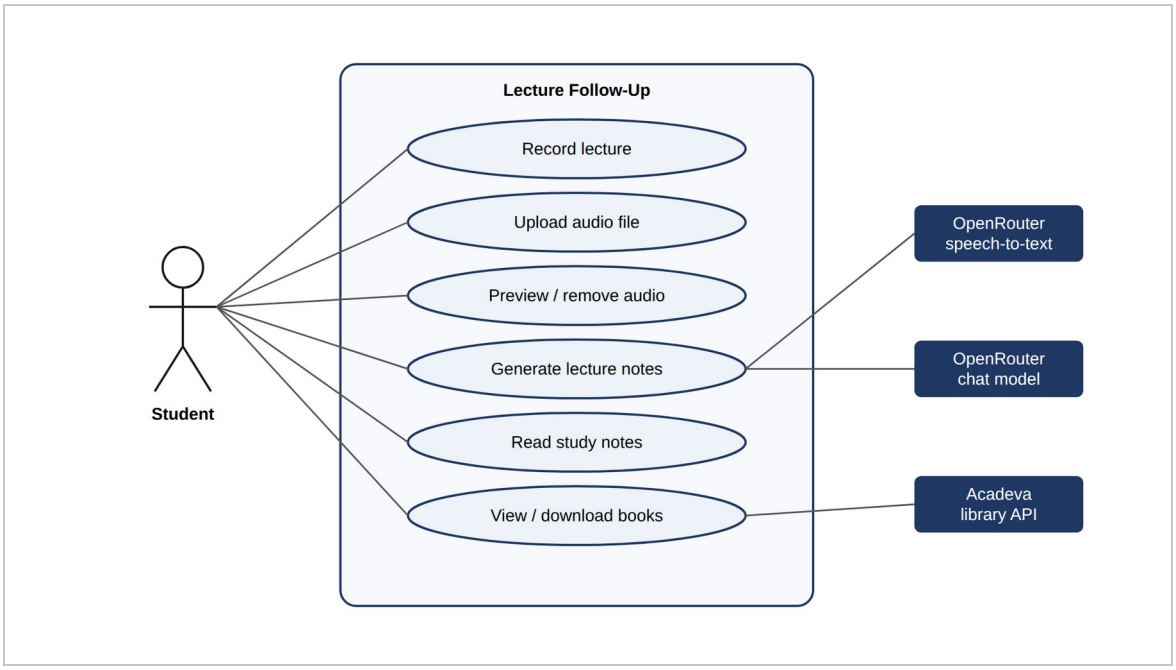


Figure 3.1: Use case diagram of Lecture Follow-Up

Table 3.3: Use case description for "Generate lecture notes"

Item	Description
Actor	Student
Precondition	Audio has been recorded or uploaded and is shown in the review card.
Main flow	1. Student clicks Generate Lecture Notes. 2. Browser sends the audio to /api/lecture. 3. Server validates the format and reports transcribing. 4. Audio is transcribed. 5. Server reports explaining; notes and search terms are generated. 6. Server reports finding-books; related books are retrieved and ranked. 7. Server sends done with the result. 8. Browser shows the notes and books.
Alternative flow	If transcription or note generation fails, the server sends an error event and the browser shows the message. If the book search fails, the notes are shown with no books.
Postcondition	The student sees study notes and related books, or a clear error message.

3.8 Methodology

The project followed an incremental and iterative development approach. The work was divided into small, usable increments, and each increment went through a short cycle of planning, design, building, testing and review before the next was started (Figure 3.2).

This suited the learning nature of the placement, because each cycle applied a newly learned concept, and it meant that a working application existed from early in the project. External services were wrapped in their own modules so they could be developed and replaced independently. The increments are listed in Table 3.4 and the technologies used are summarized in Table 3.5.

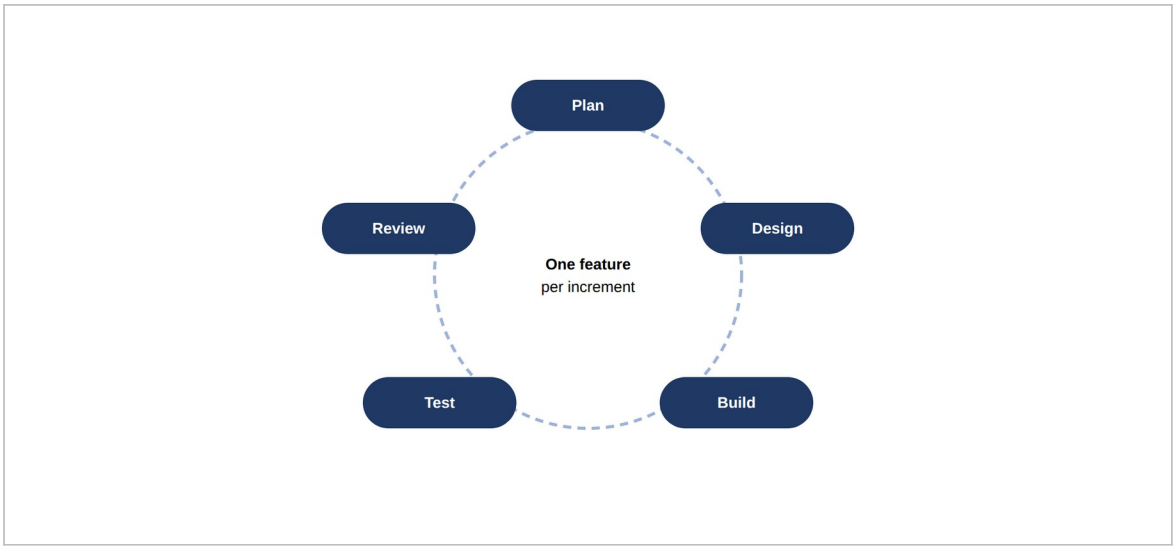


Figure 3.2: The iterative cycle applied to each increment

Table 3.4: Development increments

Increment	Outcome
1. Project setup	Next.js project with TypeScript, Tailwind CSS and shadcn/ui; base page layout.
2. Audio capture	Record tab with live waveform and Upload tab with drag-and-drop; review card with playback waveform.
3. Transcription	Route handler accepting audio, format detection and call to the speech-to-text model.
4. Note generation	System instruction, JSON schema and Markdown rendering of the notes.
5. Book recommendation	Parallel Acadeva searches, merging, ranking, fallback search and the book grid.
6. Streamed progress	NDJSON events from the server and the progress tracker in the browser.
7. Polish and testing	Error handling, responsive layout, dark mode, README, lint, build and browser tests.

Table 3.5: Technology used

Area	Technology	Purpose
User interface	React 19	Components for recording, upload, progress, notes and books
Framework	Next.js 16 (App Router)	Page routing, route handler for

Area	Technology	Purpose
		/api/lecture, production build
Language	TypeScript 5	Type-checked code and shared types for stage events and books
Styling	Tailwind CSS 4, shadcn/ui, lucide-react	Responsive layout, tabs, cards, buttons, badges and icons
Audio	wavesurfer.js 7 and Record plugin	Live recording waveform and playback waveform
Transcription	OpenRouter, openai/whisper-large-v3	Converting lecture audio to text
Note generation	OpenRouter, openai/gpt-4o-mini	Writing study notes and search terms as structured JSON
Book search	Acadeva public books API	Finding related textbooks and course materials
Rendering	react-markdown, remark-gfm, Tailwind Typography	Displaying the Markdown notes
Notifications	Sonner	Toast messages for errors
Quality	ESLint, Playwright	Static checks and browser-driven checks
Version control	Git and GitHub	History and remote hosting of the code

3.9 System Design

3.9.1 System Architecture

The system has three parts: the React interface in the browser, the Next.js route handler on the server, and three external services (OpenRouter transcription, an OpenRouter chat model and the Acadeva library). The browser never calls the AI services directly. It sends the audio to POST /api/lecture, and the server, which holds the OpenRouter key, calls each service in turn and streams progress back. Figure 3.3 shows the architecture.

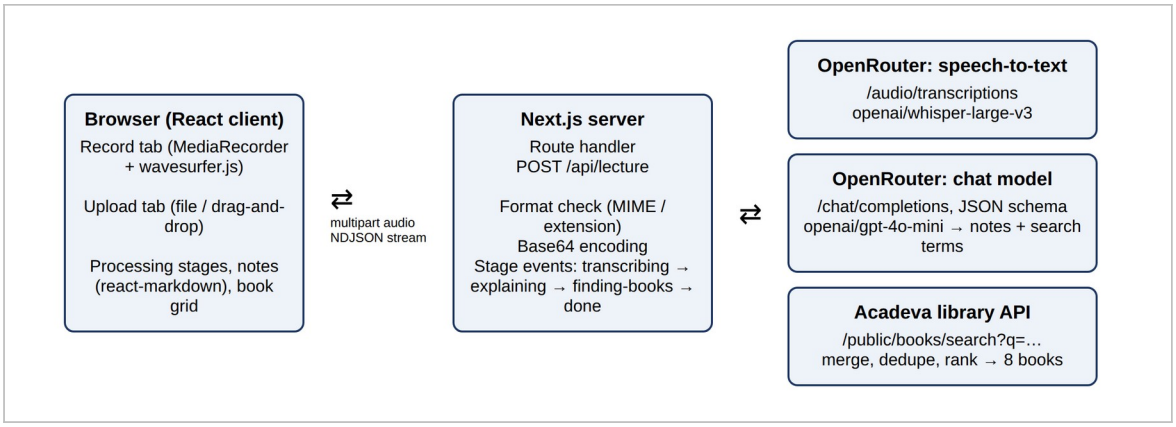


Figure 3.3: Architecture of Lecture Follow-Up

3.9.2 Sequence of Operations

Figure 3.4 shows the order of messages between the components when a student generates notes. Solid arrows are requests; dashed arrows are responses or streamed events.

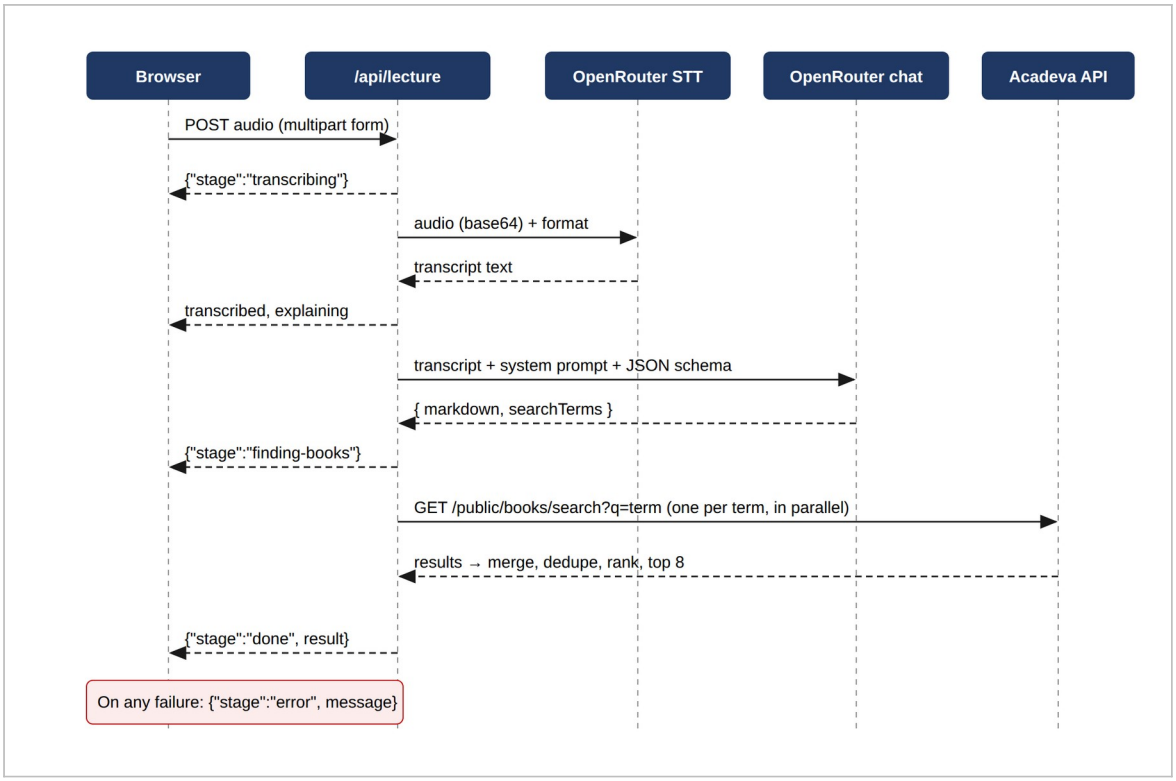


Figure 3.4: Sequence diagram for generating lecture notes

3.9.3 Interface States

The page moves through a small set of states, shown in Figure 3.5. Each state decides what the progress tracker displays and whether the Generate button is enabled. The same

state names are used in the events sent by the server, which keeps the two sides consistent.

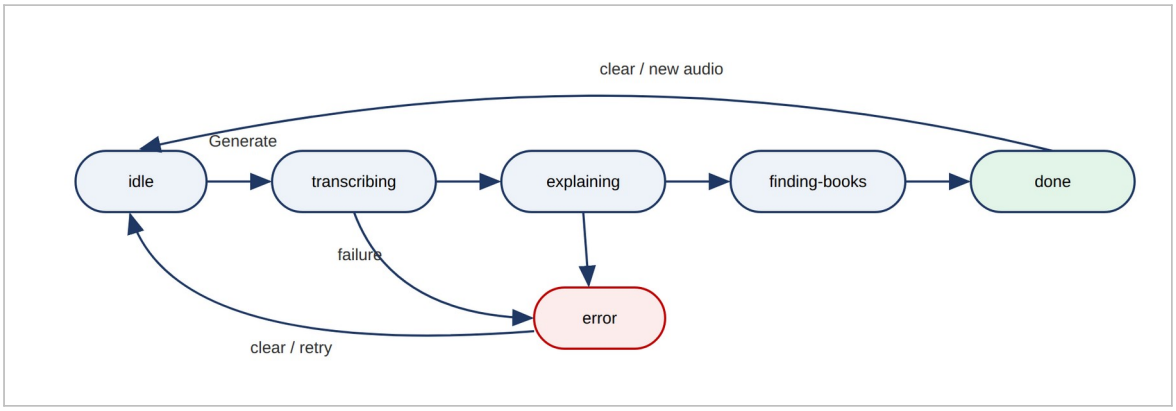


Figure 3.5: State diagram of the processing workflow

3.9.4 Book Recommendation Algorithm

The recommendation step turns the model's search terms into a short, ranked list of books. The ranking favours books that match several of the lecture's topics, and then books that other students have rated highly and downloaded often. The steps are shown in Figure 3.6.

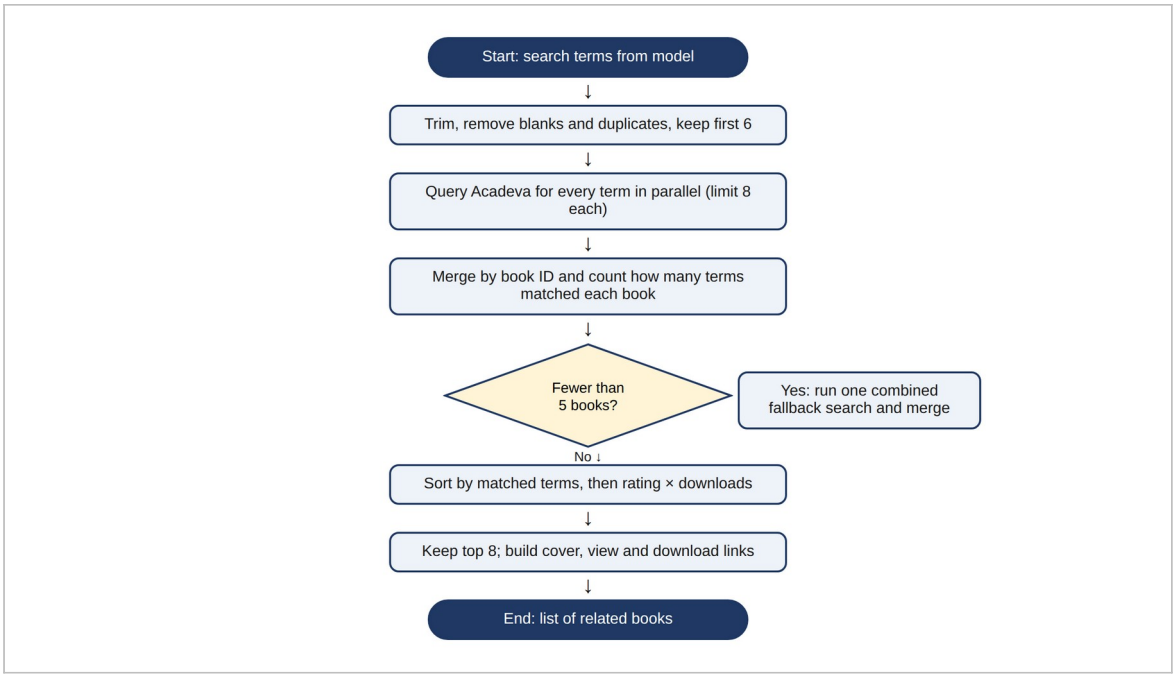


Figure 3.6: Flowchart of the book recommendation algorithm

3.9.5 API Design

The application exposes a single endpoint. Its request and response are described in Table 3.6, and the events it streams are listed in Table 3.7.

Table 3.6: Specification of the POST /api/lecture endpoint

Item	Description
Method and path	POST /api/lecture
Request body	multipart/form-data with field audio (the audio file) and optional field mimeType
Accepted formats	wav, mp3, flac, m4a, ogg, webm, aac (from MIME type, or from the file extension)
Error responses	400 {"error": "No audio file provided"}; 400 {"error": "Unsupported audio format ..."}
Success response	200, Content-Type application/x-ndjson, one JSON event per line
Maximum duration	120 seconds (maxDuration), Node.js runtime

Table 3.7: Stage events streamed from POST /api/lecture

Event	When it is sent	Content
transcribing	Before the audio is sent for transcription	Stage name only
transcribed	When transcription succeeds	The transcript text
explaining	Before the transcript is sent to the chat model	Stage name only
finding-books	Before the Acadeva search starts	Stage name only
done	When all stages finish	Markdown notes, search terms and the list of books
error	When transcription or note writing fails	A readable error message

3.9.6 Data Design

The application does not use a database; each request is processed independently. Its data is described by TypeScript types shared between the server and the browser. The Book type is shown in Table 3.8, and the stage events in Listing 3.1.

Table 3.8: Fields of the Book type

Field	Type	Description
id	string	Acadeva book identifier
title	string	Book title
description	string	Short description
rating	number	Average rating
downloads	number	Number of downloads
pages	number	Number of pages
size	number	File size in megabytes
courseCode	string (optional)	Course code, for example cit333
coverUrl	string	Cover image URL on Cloudinary

Field	Type	Description
viewUrl	string	Link to the book page on Acadeva
downloadUrl	string	Direct download link on the Acadeva CDN

Listing 3.1: Shared type for the streamed stage events (lib/types.ts)

```
/**
 * Stages streamed to the client as newline-delimited JSON while
 * POST /api/lecture processes an uploaded/recorded audio file.
 */
export type LectureStageEvent =
  | { stage: "transcribing" }
  | { stage: "transcribed"; transcript: string }
  | { stage: "explaining" }
  | { stage: "finding-books" }
  | { stage: "done"; result: LectureResult }
  | { stage: "error"; message: string };
```

3.9.7 Prompt Design

The quality of the notes depends heavily on the instruction given to the language model. The system instruction was written to ask for explanations rather than a cleaned transcript, to impose a predictable structure, and to obtain search terms suited to a library catalogue. Its main rules are summarized in Table 3.9.

Table 3.9: Rules in the system instruction for note generation

Rule	Reason
Clean up filler words, false starts and transcription errors.	Lecture audio is noisy; the notes should not inherit its mistakes.
One H1 title, then H2 sections in the order discussed.	Gives every set of notes a predictable, scannable structure.
Explain in the model's own words, define key terms, give examples.	Notes should teach, not repeat the lecture word for word.
Bold key terms on first mention; use lists for steps and comparisons.	Makes revision faster.
End with 3 to 5 Key Takeaways.	Gives a quick summary for last-minute revision.
Return 3 to 6 search phrases of 2 to 4 words, phrased like a textbook catalogue.	Produces terms that match how books are titled in the Acadeva library.

3.9.8 Module Structure

Table 3.10: Main source files and their responsibilities

File	Responsibility
app/page.tsx	Main page; holds the processing state and reads the NDJSON stream
app/api/lecture/route.ts	Route handler; validates audio and runs the three

File	Responsibility
	stages
lib/openrouter.ts	Transcription and note generation through OpenRouter
lib/acadeva.ts	Acadeva search, merging, ranking and link building
lib/types.ts	Shared types for books, results and stage events
components/lecture/recorder-panel.tsx	Record and Upload tabs, review card and Generate button
components/lecture/waveform-recorder.tsx	Live recording with wavesurfer.js and microphone selection
components/lecture/waveform-player.tsx	Playback waveform for captured audio
components/lecture/processing-stages.tsx	Progress bar, stage list and error panel
components/lecture/lecture-notes.tsx	Markdown rendering of the notes
components/lecture/book-grid.tsx, book-card.tsx	Grid of related books with view and download links

3.10 System Implementation

This section describes how the main parts of the design were implemented, with excerpts from the source code.

3.10.1 Audio Capture and Upload

The recorder panel offers two tabs. The Record tab uses the waveform recorder component, which requests microphone access, lists the available input devices and shows a scrolling waveform while recording. The Upload tab accepts a file by clicking or dragging, and rejects files whose type does not begin with audio/. Both paths end in the same captured-audio state, which replaces the tabs with a single review card containing the file name, a playback waveform, the duration and the Generate Lecture Notes button.

Listing 3.2: Accepting dropped or selected files (components/lecture/recorder-panel.tsx)

```
  const handleFile = useCallback(
    (file: File) => {
      if (!file.type.startsWith("audio/")) {
        toast.error("Please choose an audio file");
        return;
      }
      setCapturedAudio(file, file.type || "audio/mpeg", file.name, "upload");
    },
    [setCapturedAudio]
  );

  const handleDrop = useCallback(
    (e: React.DragEvent<HTMLLabelElement>) => {
      e.preventDefault();
      setIsDragging(false);
      const file = e.dataTransfer.files?.[0];
      if (file) handleFile(file);
    },
    [handleFile]
  );
```

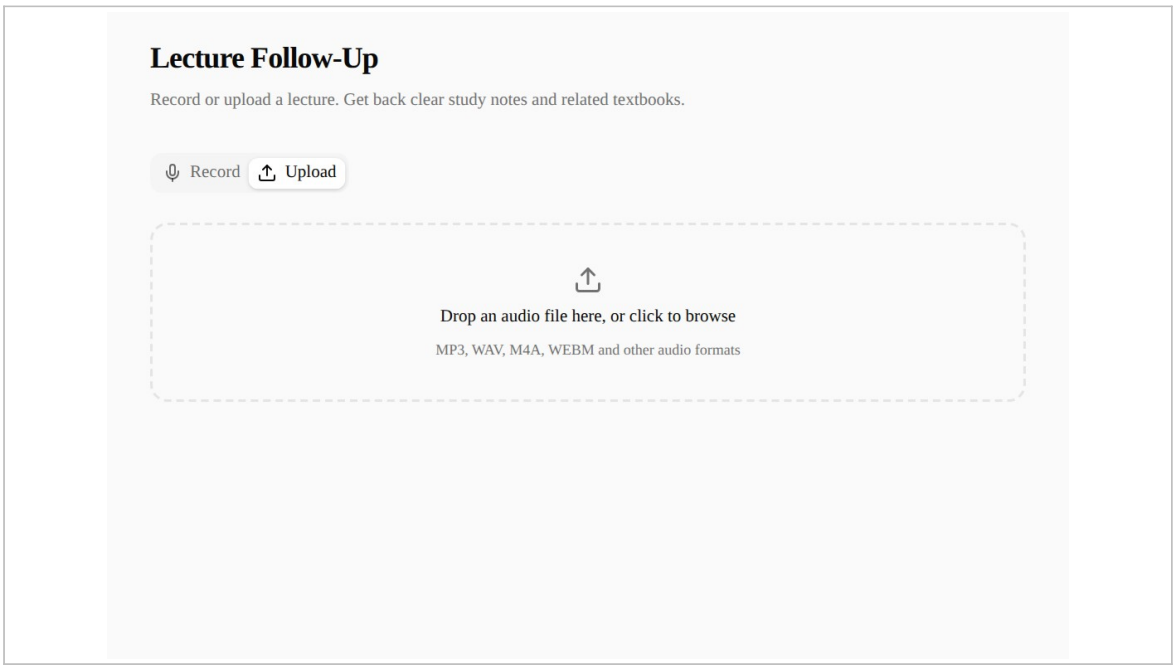


Figure 3.7: The Upload tab with the drag-and-drop area and accepted formats

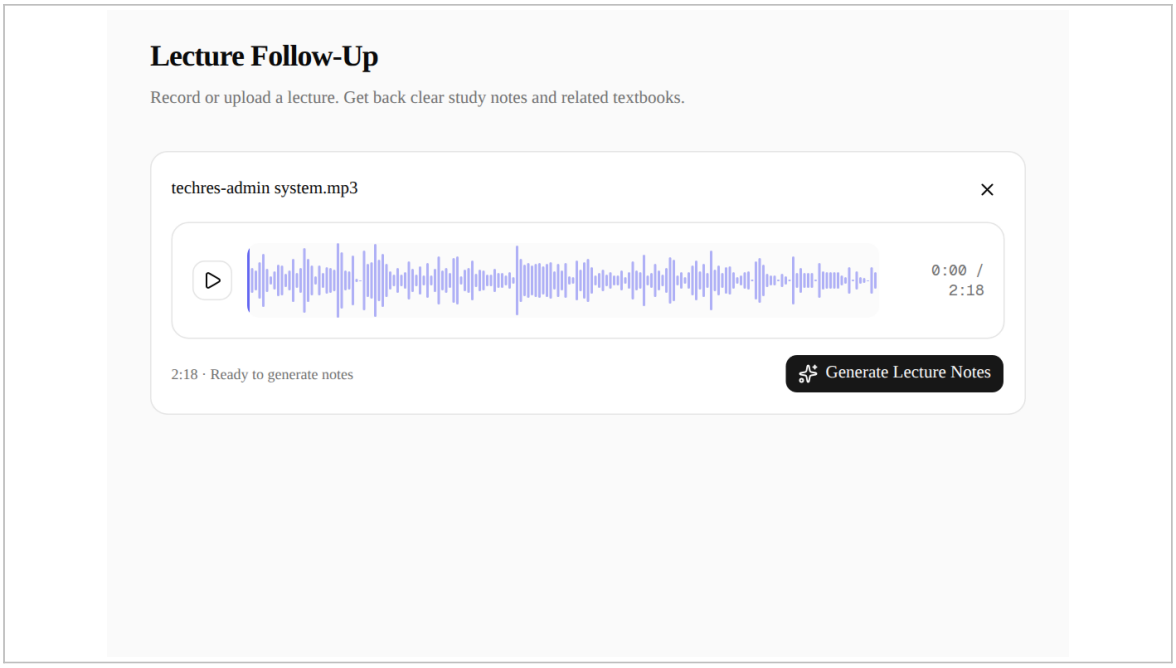


Figure 3.8: Review card showing an uploaded recording (2 minutes 18 seconds) as a playback waveform

3.10.2 The Processing Route

The route handler first validates the request, then creates a stream and runs the stages one after another inside it. Each stage sends an event before it starts, so the browser can update the tracker immediately. Failures in transcription or note generation end the stream with an error event; a failure in the book search is logged and replaced with an empty list so that the notes are not lost.

Listing 3.3: Transcription stage with error reporting (app/api/lecture/route.ts)

```
        send({ stage: "transcribing" });

        let transcript: string;
        try {
          transcript = await transcribeAudio({ audioBase64, format });
        } catch (err) {
          send({
            stage: "error",
            message: `Failed to transcribe audio: ${
              err instanceof Error ? err.message : String(err)
            }`,
          });
          controller.close();
          return;
        }

        send({ stage: "transcribed", transcript });
```

3.10.3 Transcription and Note Generation

The transcription function sends the base64 audio and its format to OpenRouter's /audio/transcriptions endpoint and returns the text. The note generation function sends the

system instruction and the transcript to /chat/completions with a strict JSON schema, then parses the reply into the markdown and searchTerms fields. Both functions throw an error containing the provider's status code and message when a request fails, which the route turns into a readable error event.

Listing 3.4: The transcription request (lib/openrouter.ts)

```
export async function transcribeAudio(params: {
  audioBase64: string;
  format: string;
}): Promise<string> {
  const model = process.env.OPENROUTER_STT_MODEL ?? "openai/whisper-large-v3";

  const res = await fetch(`${OPENROUTER_BASE_URL}/audio/transcriptions`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${process.env.OPENROUTER_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      model,
      input_audio: {
        data: params.audioBase64,
        format: params.format,
      },
    }),
  });

  if (!res.ok) {
    const body = await res.text();
    throw new Error(`Transcription request failed (${res.status}): ${body}`);
  }

  const data = (await res.json()) as { text: string; usage?: unknown };
  return data.text;
}
```

3.10.4 Book Search and Ranking

The search module implements the algorithm in Figure 3.6. Matches are counted per book, so a book that appears in the results for several topics rises to the top. Ties are broken by multiplying rating by downloads, which favours books that are both well rated and widely used.

Listing 3.5: Ranking the merged results (lib/acadeva.ts)

```
  const sorted = Array.from(bookById.values()).sort((a, b) => {
    const matchA = matchCounts.get(a.id) ?? 0;
    const matchB = matchCounts.get(b.id) ?? 0;
    if (matchB !== matchA) return matchB - matchA;

    const scoreA = a.downloads > 0 ? a.rating * a.downloads : a.rating;
    const scoreB = b.downloads > 0 ? b.rating * b.downloads : b.rating;
    return scoreB - scoreA;
  });

  return sorted.slice(0, MAX_RESULTS).map(mapBook);
```

3.10.5 Progress Display and Results

The progress tracker shows a progress bar and the three stages, each with an icon for pending, active or done. When the done event arrives, the notes are rendered with react-markdown and the Tailwind Typography styles, followed by the book grid. Figures 3.9 to 3.12 show these screens.

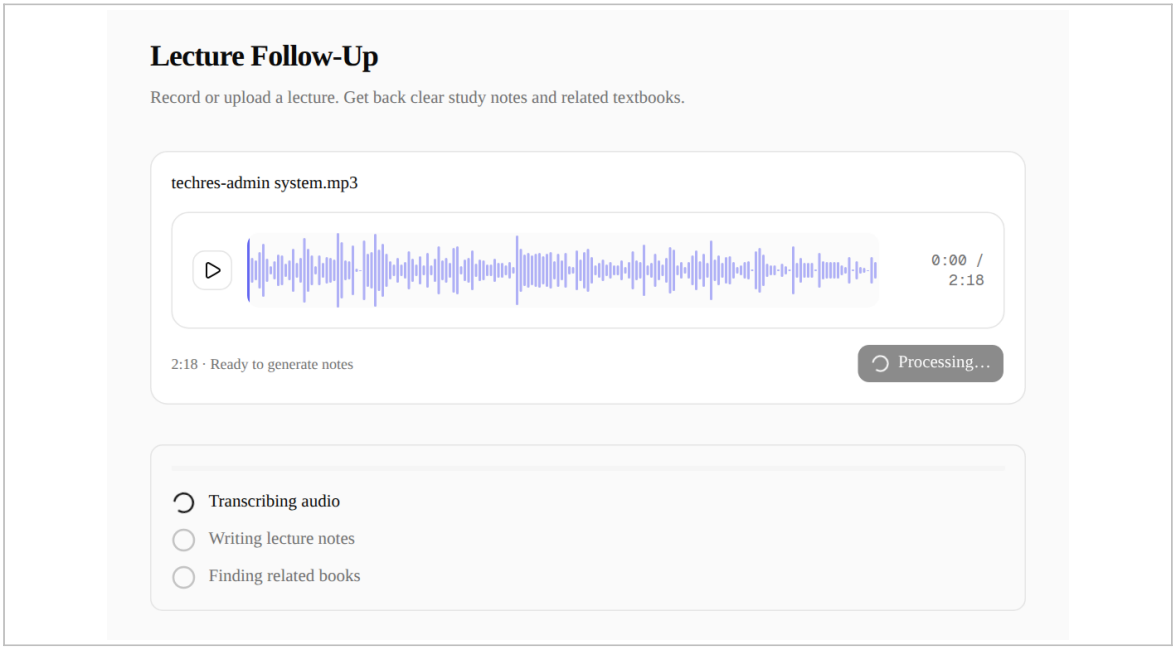


Figure 3.9: Progress tracker while the lecture is being transcribed

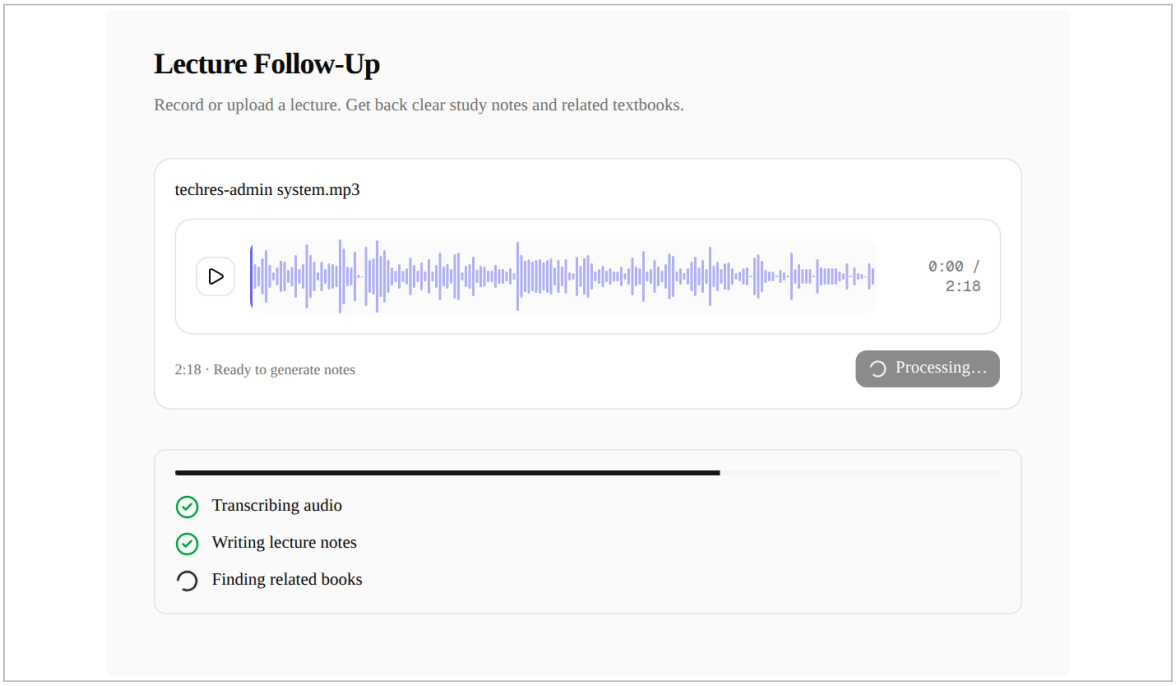
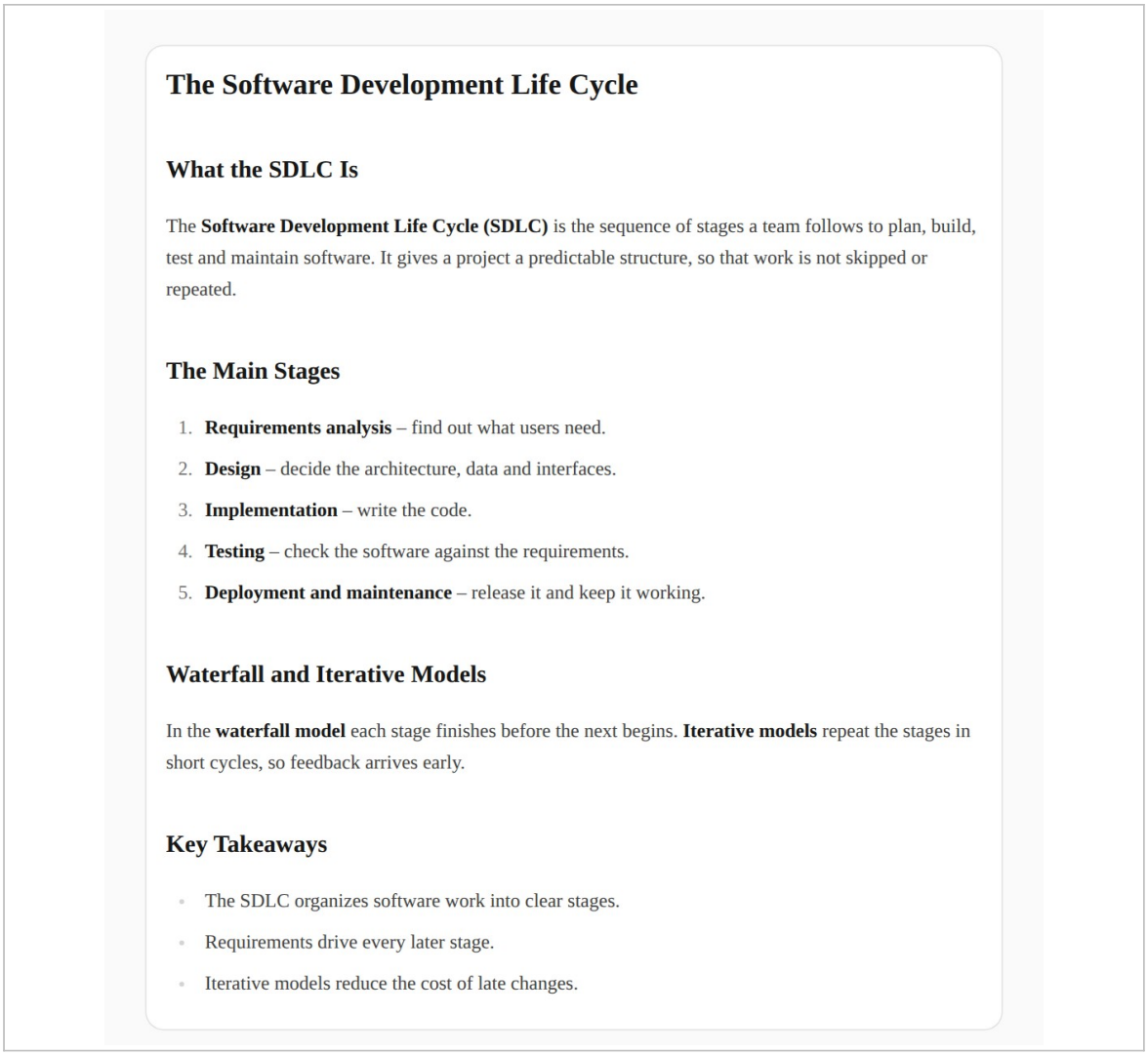


Figure 3.10: Progress tracker after transcription and note writing, while related books are being found



The Software Development Life Cycle

What the SDLC Is

The **Software Development Life Cycle (SDLC)** is the sequence of stages a team follows to plan, build, test and maintain software. It gives a project a predictable structure, so that work is not skipped or repeated.

The Main Stages

1. **Requirements analysis** – find out what users need.
2. **Design** – decide the architecture, data and interfaces.
3. **Implementation** – write the code.
4. **Testing** – check the software against the requirements.
5. **Deployment and maintenance** – release it and keep it working.

Waterfall and Iterative Models

In the **waterfall model** each stage finishes before the next begins. **Iterative models** repeat the stages in short cycles, so feedback arrives early.

Key Takeaways

- The SDLC organizes software work into clear stages.
- Requirements drive every later stage.
- Iterative models reduce the cost of late changes.

Figure 3.11: Completed stages and the rendered study notes (the notes shown are sample content supplied to the interface for illustration)

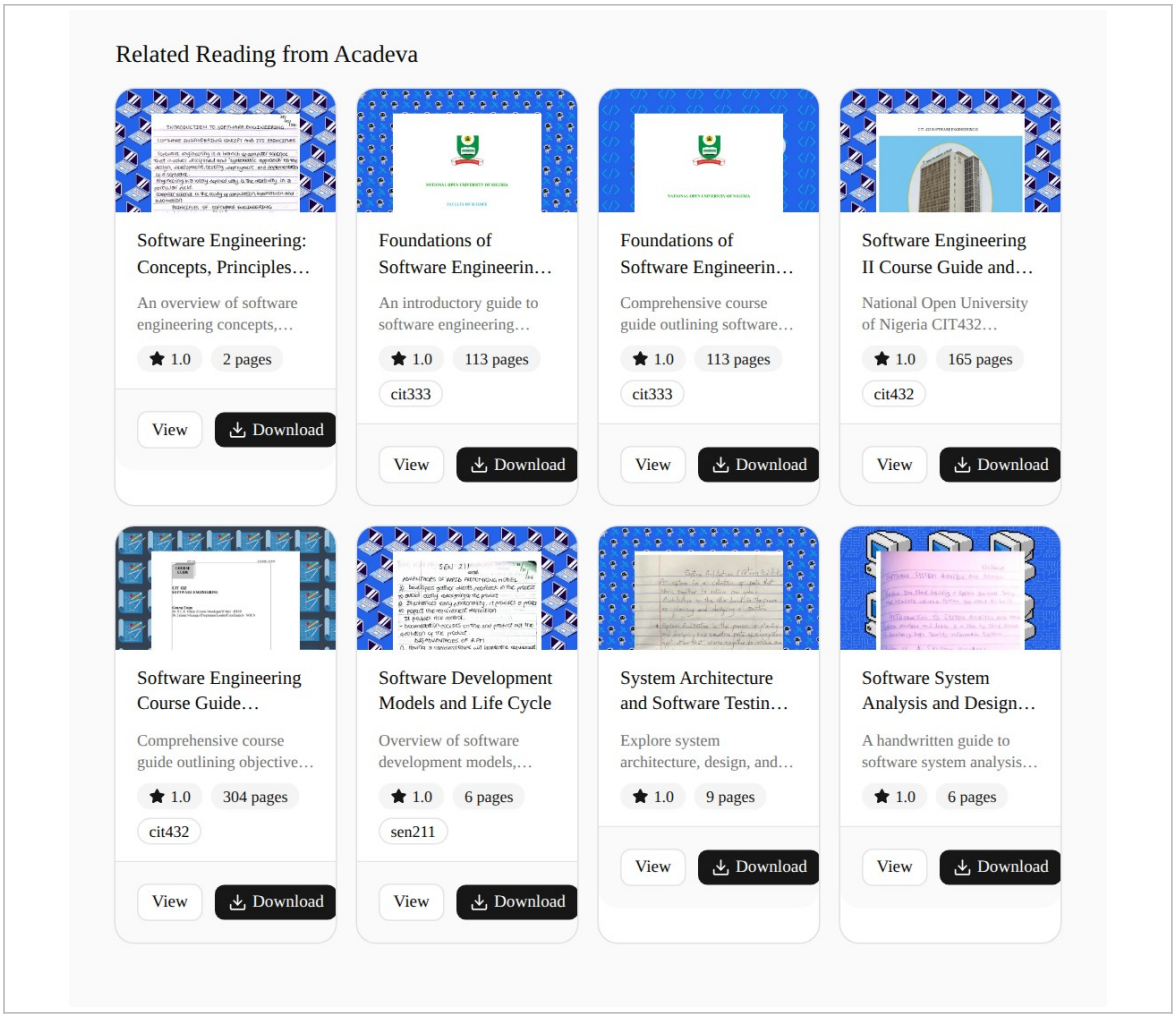


Figure 3.12: Related reading from the Acadeva library, returned by a live search for "software engineering"

3.10.6 Error Handling

Every external call can fail: the network may drop, the provider may reject the key or the balance may be too low. Each failure produces a specific message that names the stage that failed and includes the provider's reason. The message is shown in a red panel under the review card and as a toast notification, and the student can remove the audio and try again. Figure 3.13 shows the result of submitting a recording when no API key is configured.



Figure 3.13: Error panel shown when the transcription service rejects the request

3.10.7 Responsive Design

The page uses a single centred column with a maximum width, which narrows naturally on phones. The book grid shows two columns on phones, three on tablets and four on desktops. Figure 3.14 shows the application on an emulated Pixel 7 phone.

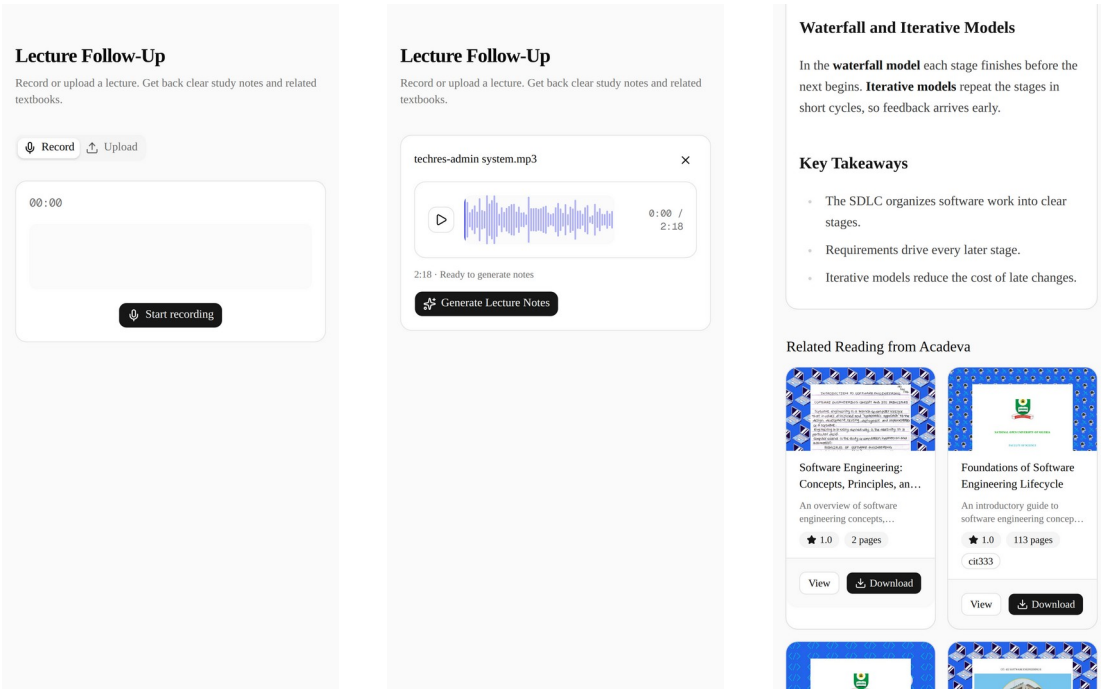


Figure 3.14: Lecture Follow-Up on a phone: home page (left), uploaded recording (centre) and related books (right)

3.10.8 Configuration

Table 3.11: Environment variables

Variable	Required	Default	Purpose
OPENROUTER_KEY	Yes	None	OpenRouter API key; audio requests need at least $0.50 of available balance
OPENROUTER_STT_MODEL	No	openai/whisper-large-v3	Speech-to-text model
OPENROUTER_EXPLAIN_MODEL	No	openai/gpt-4o-mini	Chat model that supports strict JSON schema output

3.11 Testing

3.11.1 Test Strategy

Testing was carried out at four levels. **Static checks:** ESLint was run over the whole code base and reported no errors, and a production build with next build completed successfully, which also type-checks the TypeScript code. **API tests:** the endpoint was called directly with curl using missing, invalid and valid input. **Browser tests:** the interface was driven in Chromium with Playwright at 1280 by 860 pixels and on an emulated Pixel 7 phone, using the sample recordings in the testfiles folder and a simulated microphone; for the progress and results screens the network response was supplied by the test so that each state could be checked on its own. **External service checks:** the Acadeva search endpoint was queried directly to confirm that it returns real books for typical search terms.

3.11.2 Test Cases

Table 3.12: Test cases and results

ID	Test	Expected result	Result
T1	Run pnpm lint	No lint errors	Pass
T2	Run pnpm build	Build completes; / is static and /api/lecture is dynamic	Pass
T3	POST /api/lecture with no audio field	HTTP 400, "No audio file provided"	Pass
T4	POST a plain text file as audio	HTTP 400 naming the format and listing accepted formats	Pass
T5	Open the home page	Record and Upload tabs and Start recording button shown	Pass

ID	Test	Expected result	Result
T6	Record with a simulated microphone, then stop	Live waveform and timer while recording; review card with recording.webm afterwards	Pass
T7	Upload techres-admin system.mp3	Playback waveform with duration 2:18 and Generate button	Pass
T8	Click Generate while the request is in progress	Button shows Processing…; tracker shows Transcribing audio	Pass
T9	Server sends explaining and finding-books events	First two stages ticked; third stage active; progress bar advances	Pass
T10	Submit audio when the transcription service rejects the key	Stream ends with error event; red panel and toast show the reason	Pass
T11	Server sends done with notes and books	Notes rendered as formatted Markdown; books in a grid with View and Download	Pass
T12	Search Acadeva for "software engineering"	Eight related books returned with covers and course codes	Pass
T13	Open the page on a Pixel 7 screen size	Layout fits the screen without horizontal scrolling; two-column book grid	Pass
T14	Open the page with the system in dark mode	Interface switches to dark colours	Pass

3.12 Evaluation and Discussion of Results

The application was evaluated against the requirements in Section 3.7 using the following factors:

- **Correctness:** every functional requirement (FR1 to FR9) was demonstrated by at least one test case in Table 3.12.
- **Security:** the API key is read only on the server; input is validated before any external call; error messages report the provider's reason without exposing secrets.
- **Usability:** a lecture can be turned into notes with two actions (capture audio, then Generate), and progress is visible throughout.
- **Robustness:** each external call is handled separately, so one failure produces a clear message, and a book-search failure does not lose the notes.
- **Maintainability:** services are isolated in lib/openrouter.ts and lib/acadeva.ts, models are configurable, and types are shared between server and client.

The streamed progress display proved to be the most important usability decision: processing a lecture involves three slow external calls, and seeing each one complete gives the user confidence that the application is working. The strict JSON schema removed a whole class of parsing failures, and the parallel, ranked book search returned relevant materials even when individual search terms were too narrow. The main

dependency of the system is the AI provider: without a funded key the application cannot produce notes, although it fails clearly and safely.

3.13 Recommendations and Future Enhancements

Based on the work done, the following improvements are recommended:

- Add user accounts so students can save, search and revisit their notes and recordings.
- Allow notes to be edited and exported to PDF, or shared with classmates through Acadeva.
- Split long recordings into chunks so that lectures of an hour or more can be processed reliably.
- Generate quizzes and flashcards from the notes, linking to Acadeva's existing quiz features.
- Match recommendations to the student's own school and course code when they are known.
- Add automated unit and end-to-end tests to the repository and run them on every pull request with GitHub Actions.
- Improve accessibility, including keyboard control of the recorder and screen reader announcements for each stage.
- Offer the feature inside the Acadeva mobile and desktop apps.

3.14 Limitations of the Study

The project was built within a six-month, learning-focused placement, which limited its scope. It depends on paid third-party services: transcription and note generation require an OpenRouter key with available credit, and the quality of the notes depends on the clarity of the recording and on the chosen models, which can still make mistakes, so students should check important facts against their textbooks. Very long recordings may exceed the request limits of the transcription service. Notes are not stored, so they are lost when the page is closed. The system was tested with sample recordings rather than in a formal study with students, and no load testing was carried out.

3.15 Conclusion

The project designed and implemented Lecture Follow-Up, a web application that turns a recorded or uploaded lecture into structured study notes and recommends related textbooks from the Acadeva library, using React, Next.js, OpenRouter AI models and the Acadeva API. It showed how a browser interface, a server route and several external

services can be combined into one dependable flow. The main lessons were the importance of keeping secrets on the server, asking AI models for structured output, handling every external failure explicitly, and giving users continuous feedback during slow operations.

The SIWES placement at Acadeva Limited as a whole provided valuable exposure to professional software development, from setting up a development environment to delivering and documenting a working product. The skills and experience gained have strengthened the author's understanding of Computer Science and provided a firm foundation for final-year studies and a career in software engineering.

REFERENCES

Acadeva (n.d.) Acadeva. Available at: https://acadeva.xyz/ (Accessed: 4 October 2026).

Git (n.d.) Git documentation. Available at: https://git-scm.com/doc (Accessed: 4 October 2026).

GitHub (n.d.) GitHub Docs. Available at: https://docs.github.com/ (Accessed: 4 October 2026).

Industrial Training Fund (n.d.) Students Industrial Work Experience Scheme (SIWES). Available at: https://itf.gov.ng/ (Accessed: 4 October 2026).

Meta Open Source (n.d.) React documentation. Available at: https://react.dev/ (Accessed: 4 October 2026).

Microsoft (n.d.) Playwright documentation. Available at: https://playwright.dev/docs/intro (Accessed: 4 October 2026).

Microsoft (n.d.) TypeScript documentation. Available at: https://www.typescriptlang.org/docs/ (Accessed: 4 October 2026).

MDN Web Docs (n.d.) MediaRecorder API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder (Accessed: 4 October 2026).

MDN Web Docs (n.d.) Streams API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Streams_API (Accessed: 4 October 2026).

OpenRouter (n.d.) OpenRouter API documentation. Available at: https://openrouter.ai/docs (Accessed: 4 October 2026).

Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2023) 'Robust speech recognition via large-scale weak supervision', Proceedings of the 40th International Conference on Machine Learning (ICML), pp. 28492–28518.

shadcn (n.d.) shadcn/ui documentation. Available at: https://ui.shadcn.com/docs (Accessed: 4 October 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Boston: Pearson.

Tailwind Labs (n.d.) Tailwind CSS documentation. Available at: https://tailwindcss.com/docs (Accessed: 4 October 2026).

Vercel (n.d.) Next.js documentation. Available at: https://nextjs.org/docs (Accessed: 4 October 2026).

wavesurfer.js (n.d.) wavesurfer.js documentation. Available at: https://wavesurfer.xyz/ (Accessed: 4 October 2026).

APPENDICES

Appendix A: Installing and Running the Project

The project requires Node.js 22 and pnpm. After cloning the repository from GitHub, it is installed and run with the commands below. A file named .env must contain at least OPENROUTER_KEY (see Table 3.11).

Listing A.1: Commands for installing and running Lecture Follow-Up

```
git clone https://github.com/emmanuelekopimo/lecture-follow-up.git
cd lecture-follow-up
pnpm install

# development server on http://localhost:3000
pnpm dev

# production build and server
pnpm build
pnpm start

# static checks
pnpm lint
```

Appendix B: Sample Recordings

Table B.1: Sample recordings in the testfiles folder

File	Size	Use
Welcome back to Square One.mp3	1.0 MB	Manual testing of the upload path
techres-admin system.mp3	1.9 MB (2 min 18 s)	Upload, progress and results screens in this report
ecews-report.mp3	7.7 MB	Testing a larger upload and the error path

Appendix C: Selected Source Code

Listing C.1: Searching, merging and falling back (lib/acadeva.ts)

```
export async function searchBooks(terms: string[]): Promise<Book[]> {
  const cleanedTerms = Array.from(
    new Set(terms.map((t) => t.trim()).filter((t) => t.length > 0))
  ).slice(0, MAX_TERMS);

  if (cleanedTerms.length === 0) {
    return [];
  }

  const matchCounts = new Map<string, number>();
  const bookById = new Map<string, RawBook>();

  const mergeResults = (rawBooks: RawBook[]) => {
    const seenInThisSearch = new Set<string>();
    for (const raw of rawBooks) {
      if (seenInThisSearch.has(raw.id)) continue;
      seenInThisSearch.add(raw.id);
      if (!bookById.has(raw.id)) {
        bookById.set(raw.id, raw);
      }
      matchCounts.set(raw.id, (matchCounts.get(raw.id) ?? 0) + 1);
    }
  };

  const perTermResults = await Promise.all(
    cleanedTerms.map((term) => fetchTerm(term, 8))
  );
  for (const results of perTermResults) {
    mergeResults(results);
  }

  if (bookById.size < 5) {
    const fallbackTerm = cleanedTerms.join(" ").slice(0, 60);
    const fallbackResults = await fetchTerm(fallbackTerm, 10);
    mergeResults(fallbackResults);
  }
```

Listing C.2: Rendering the notes with react-markdown (components/lecture/lecture-notes.tsx)

```
export function LectureNotes(props: LectureNotesProps) {
  return (
    <Card>
      <CardContent
        className={[
          "prose prose-neutral dark:prose-invert max-w-none",
          "prose-headings:scroll-mt-20 prose-headings:font-heading",
          "prose-h1:text-2xl prose-h2:text-xl prose-h3:text-lg",
          "prose-p:leading-relaxed prose-li:leading-relaxed",
          "prose-pre:bg-muted prose-pre:text-foreground",
          "prose-code:before:content-none prose-code:after:content-none",
          "prose-table:text-sm",
        ].join(" ")}
      >
        <ReactMarkdown remarkPlugins={[remarkGfm]}>
          {props.markdown}
        </ReactMarkdown>
      </CardContent>
    </Card>
  )
}
```

Listing C.3: The book grid (components/lecture/book-grid.tsx)

```
export function BookGrid(props: BookGridProps) {
  const { books } = props

  return (
    <section className="space-y-3">
      <h2 className="text-lg font-medium">Related Reading from Acadeva</h2>
      {books.length === 0 ? (
        <p className="text-sm text-muted-foreground">
          No related books found.
        </p>
      ) : (
        <div className="grid grid-cols-2 gap-4 sm:grid-cols-3 lg:grid-cols-4">
          {books.map((book) => (
            <BookCard key={book.id} book={book} />
          ))}
        </div>
      )}
    </section>
  )
}
```

Appendix D: Additional Screenshots

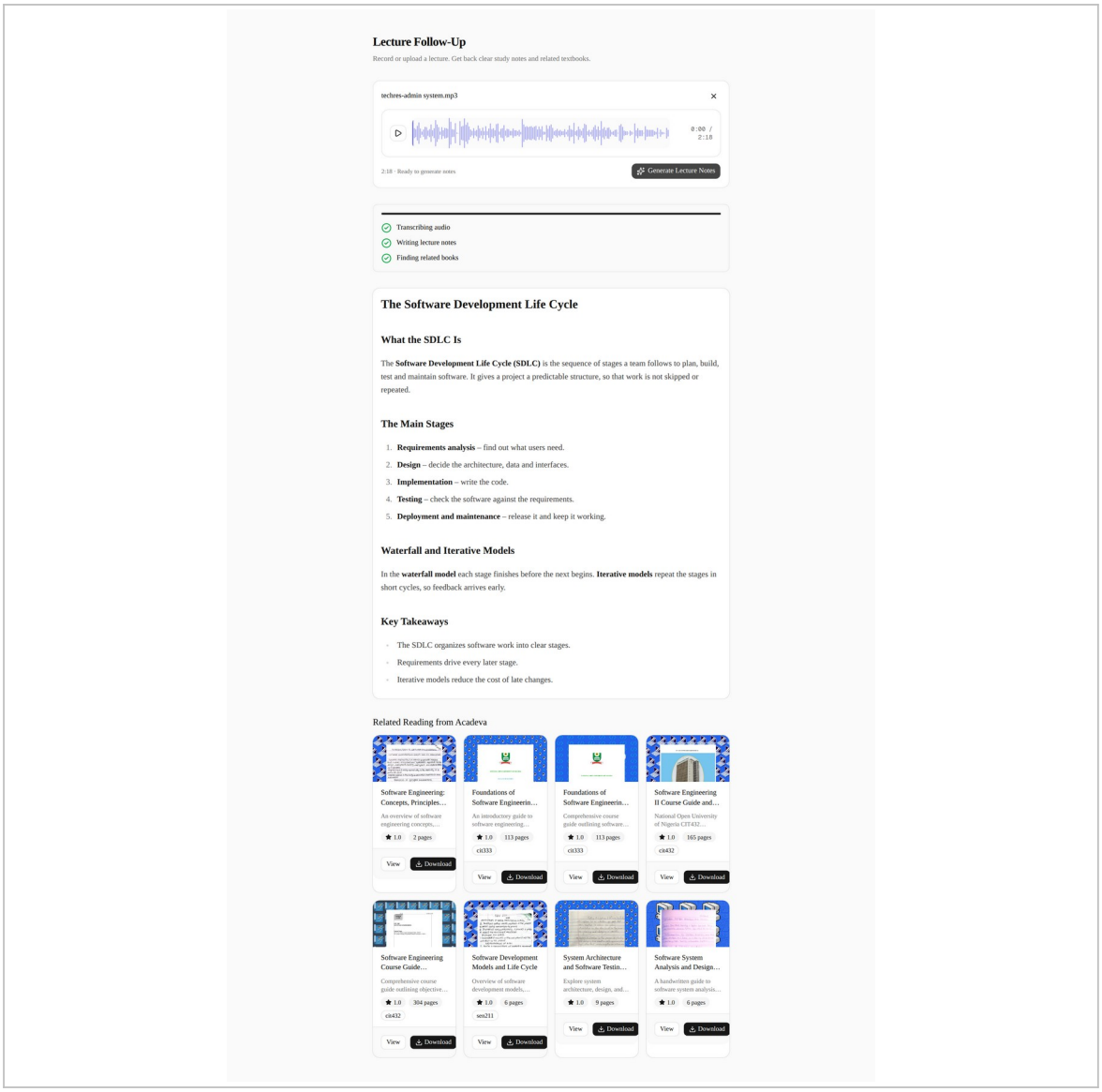


Figure D.1: Full results page on desktop: review card, completed stages, notes and related books